

UNIVERZITA KOMENSKÉHO V BRATISLAVE
FAKULTA MATEMATIKY, FYZIKY A INFORMATIKY

GRAFOVÉ EDITORY PRE PRIESKUMNÍK
ŠTRUKTÚR LOGIKY PRVÉHO RÁDU
BAKALÁRSKA PRÁCA

UNIVERZITA KOMENSKÉHO V BRATISLAVE
FAKULTA MATEMATIKY, FYZIKY A INFORMATIKY

GRAFOVÉ EDITORY PRE PRIESKUMNÍK
ŠTRUKTÚR LOGIKY PRVÉHO RÁDU
BAKALÁRSKA PRÁCA

Študijný program: Aplikovaná informatika
Študijný odbor: Informatika
Školiace pracovisko: Katedra aplikovanej informatiky
Školiteľ: Mgr. Ján Kľuka, PhD.



ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Jakub Marček
Študijný program: aplikovaná informatika (Jednoodborové štúdium, bakalársky I. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: bakalárska
Jazyk záverečnej práce: slovenský
Sekundárny jazyk: anglický

Názov: Grafové editory pre prieskumník štruktúr logiky prvého rádu
Graph editors for the first-order logic structure explorer

Anotácia: V rámci série predchádzajúcich bakalárskych prác (Cifra, 2018; Baluch, 2020; Tóth, 2021) vznikla interaktívna webová aplikácia, ktorá umožňuje skúmať pravdivosť formúl a hodnoty termov v konkrétnych konečných štruktúrach pre jazyky logiky prvého rádu. Vnútorň návrh aplikácie však vznikol v čase, keď použité frameworky neposkytovali súčasné prostriedky, je neprehľadný a ďalšie úpravy sú prácne a prinášajú riziko vzniku ťažko odhaliteľných chýb. Naš bakalársky študent v súčasnosti pracuje na reimplementácii. Ďalším krokom vo vývoji je poskytnúť možnosť editovať interpretácie predikátov a funkcií v grafovej forme (orientované a bipartitné grafy, Hasseho diagramy a pod.)

Cieľ:

- Vytvoriť prehľad rôznych foriem grafovej reprezentácie relácií a funkcií v diskkrétnej matematike.
- Vybrať vhodnú knižnicu pre jazyk TypeScript podporujúcu interaktívnu prácu s grafmi.
- Implementovať komponenty na zobrazovanie a interaktívnu úpravu vybraných grafových reprezentácií.
- Integrovať tieto komponenty do aplikácie Prieskumník štruktúr.

Literatúra: Cifra, Milan. Prieskumník sémantiky logiky prvého rádu. Bakalárska práca. Bratislava : Univerzita Komenského, 2018.
Baluch, Miroslav. Prieskumník grafových štruktúr pre logiku prvého rádu. Bakalárska práca. Bratislava : Univerzita Komenského, 2020.
Tóth, Richard. Henkinova-Hintikkova hra v prieskumníku štruktúr. Bakalárska práca. Bratislava : Univerzita Komenského, 2021.
Bieliková, Silvia. Generovanie používateľského rozhrania z grafových dát. Bakalárska práca. Bratislava : Univerzita Komenského, 2024.

Vedúci: Mgr. Ján Kľuka, PhD.
Katedra: FMFI.KAI - Katedra aplikovanej informatiky
Vedúci katedry: doc. RNDr. Tatiana Jajcayová, PhD.
Dátum zadania: 28.10.2024

Dátum schválenia: 18.03.2025

doc. RNDr. Damas Gruska, PhD.
garant študijného programu



Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

.....
šstudent

.....
vedúci práce

Podakovanie: Ďakujem.

Abstrakt

V práci sa zaoberáme vývojom nových nástrojov rozširujúcich existujúcu webovú aplikáciu Prieskumník štruktúr logiky prvého rádu. Hlavné rozšírenie spočíva v troch rôznych interaktívnych grafových editoroch, ktoré umožňujú vizualizovať a upravovať interpretácie predikátov a funkcií vo forme orientovaného grafu, bipartitného grafu a Hasseho diagramu. Zaoberáme sa návrhom, ako premeniť jednotlivé reprezentácie do prehľadnej interaktívnej podoby, a aj ich následnou implementáciou a integráciou do existujúcej aplikácie. Podobným spôsobom tiež predstavujeme niekoľko ďalších nástrojov a vylepšení. Novú verziu aplikácie, zahŕňajúcu všetku pridanú funkcionality, sme následne integrovali do Logického pracovného zošita používaného na predmete Logika pre informatikov. V závere sa venujeme vyhodnoteniu testovania použiteľnosti našich riešení, ktoré prebehlo so študentmi predmetu.

Kľúčové slová: logika prvého rádu, grafový editor, webová aplikácia

Abstract

TODO

Keywords: first-order logic, graph editor, web application

Obsah

Úvod	1
1 Základné pojmy	3
1.1 Logika prvého rádu	3
1.1.1 Syntax	3
1.1.2 Sémantika	4
1.2 Grafové reprezentácie	5
1.2.1 Orientovaný graf	5
1.2.2 Hasseho diagram	6
1.2.3 Bipartitný graf	7
1.3 Použité technológie	8
1.3.1 React	8
1.3.2 TypeScript	8
1.3.3 React Flow	8
1.3.4 Redux	9
2 Súvisiace práce	11
2.1 Prieskumník štruktúr	11
2.2 Prieskumník grafových štruktúr	13
2.3 Logický pracovný zošit	15
3 Návrh	17
3.1 Požiadavky	17
3.1.1 Zámer rozšírenia aplikácie	17
3.1.2 Požiadavky na grafové editory	18
3.1.3 Ďalšie požiadavky	18
3.2 Spoločné rozhranie editorov	19
3.2.1 Filtrovanie	20
3.3 Grafové editory	20
3.3.1 Orientovaný graf	21
3.3.2 Hasseho diagram	22

3.3.3	Bipartitný graf	23
3.4	Tabuľkové editory	25
3.4.1	Maticový editor	25
3.4.2	Databázový editor	26
3.5	Editor interpretácií funkcií rozborom prípadov	26
3.6	Dopyty	30
3.7	Nový stav aplikácie	32
3.7.1	Spoločné rozhranie editorov	32
3.7.2	Textový pohľad	32
3.7.3	Grafové editory	34
3.7.4	Maticový a databázový editor	34
3.7.5	Editor interpretácií funkcií rozborom prípadov	34
3.7.6	Dopyty	35
4	Implementácia	37
4.1	Výber technológií	37
4.2	Integrácia nových editorov interpretácií	38
4.2.1	Organizácia	38
4.2.2	Integrácia komponentov editorov	38
4.2.3	Synchronizácia stavu	38
4.3	Implementácia grafových editorov	40
4.3.1	Štruktúra stavu	40
4.3.2	Práca so stavom	40
4.3.3	Komponenty	41
4.4	RefaktORIZÁCIA stavu pôvodnej aplikácie	42
4.5	Integrácia s Logickým pracovným zošitom	43
4.6	Vylepšenia Henkinovej-Hintikkovej hry	44
5	Testovanie	47
	Záver	51
	Príloha A	55

Úvod

Možnosti využitia elektronických nástrojov pri výučbe sú v dnešnej dobe široké a oproti konvenčným učebným metódam dokážu poskytnúť interaktívnejší a pútavejší pohľad na skúmanú problematiku. Ich tvorba však predstavuje výzvu, ktorá vyžaduje nielen znalosť danej témy, ale aj schopnosť hľadať kreatívne a inovatívne riešenia s ňou spojených problémov. V mojej práci sa budem zaoberať návrhom a implementáciou takýchto nástrojov so zámerom rozšíriť funkcionality existujúcej aplikácie.

Prvá verzia webovej aplikácie, z ktorej táto práca vychádza, vznikla v rámci bakalárskej práce Milana Cifru [12]. Jej úlohou bolo ponúknuť študentom predmetu Logika pre informatikov interaktívny nástroj na tvorbu štruktúr pre jazyky logiky prvého rádu, v ktorých je následne možné skúmať pravdivosť formúl a hodnoty termov. Neskôr bola aplikácia rozšírená v bakalárskej práci Richarda Tótha [21] o Henkinovu-Hintikkovu hru a v práci Miroslava Balucha o jeden grafový pohľad so zámerom poskytnúť študentom alternatívny spôsob vizualizácie a manipulácie týchto štruktúr. Implementácia aplikácie sa však časom stala neaktuálnou a ťažko udržiateľnou.

V bakalárskej práci Jozefa Filipa [13] bola aplikácia nanovo implementovaná s využitím aktuálnych technológií. Nová verzia však neponúka grafový ani iné pohľady poskytované pôvodnou aplikáciou. Jediný spôsob, akým je možné s ňou interagovať, je pomocou textových vstupov, ktoré sa často stávajú neprehľadnými, najmä pri väčších interpretáciách predikátových a funkčných symbolov. Taktiež neumožňujú jednoduchú vizualizáciu niektorých zaujímavých vlastností samotnej interpretácie, akou je napríklad to, že tvorí čiastočné usporiadanie.

Primárnym cieľom tejto práce je vytvoriť grafové nástroje, ktoré umožnia prehľadnejšiu manipuláciu a vizualizáciu interpretácií predikátov a funkcií. Nechceme však reimplementovať pôvodný grafový pohľad, keďže ten sa z dôvodu viacerých problémov ukázal byť medzi študentmi málo využívaný. Jednou zo zásadných zmien bude využitie troch rôznych grafových reprezentácií (orientovaný graf, bipartitný graf a Hasseho diagram), pričom každá z nich poskytne študentovi unikátny pohľad na tú istú interpretáciu. Taktiež umožníme vizualizáciu ďalších vzťahov v rámci štruktúry týkajúcich sa unárnych predikátov a individuových konštánt. Dôležitým aspektom bude poskytnúť rozhranie, ktoré je intuitívne, prehľadné a najmä jednoduché na použitie.

Prieskumník štruktúr však rozšírime aj o ďalšie nástroje. Jedným z nich bude

reimplementácia maticového a databázového pohľadu z prvej verzie aplikácie. Ďalej pribudne nástroj na jednoduchšie definovanie interpretácií funkcií pomocou rozboru prípadov. Tiež sa zameriame na rozšírenie umožňujúce zadávať dopyty na splnenie otvorených formúl v rámci vytvorenej štruktúry.

V prvej kapitole zdefinujeme základné pojmy a v krátkosti predstavíme technológie používané ďalej v práci. Druhá kapitola slúži na bližšie predstavenie súvisiacich aplikácií spolu s rozborom ich nedostatkov. Na začiatku tretej kapitoly zdefinujeme požiadavky kladené na výslednú aplikáciu. Následne sa v návrhu pozrieme na spôsob, akým sme sa rozhodli stanovené ciele dosiahnuť. Postupne opíšeme návrh jednotlivých grafových editorov, maticového a databázového editora, editora interpretácií funkcií rozborom prípadov a novej časti aplikácie venujúcej sa dopytom na splniteľnosť otvorených formúl. Tiež opíšeme návrh stavu všetkých pridaných rozšírení. V štvrtej kapitole priblížime vybrané časti implementácie. Posledná, piata kapitola rozoberá výsledky testovania našej aplikácie študentmi predmetu Logika pre informatikov.

Kapitola 1

Základné pojmy

Táto kapitola slúži na oboznámenie čitateľa s pojmami a konceptmi používanými v ďalších častiach práce. Jednotlivé pojmy do potrebnej miery vysvetlíme a pri grafových reprezentáciách aj stručne naznačíme motiváciu za ich výberom s cieľom pomôcť k lepšiemu pochopeniu ich potenciálnych prínosov.

1.1 Logika prvého rádu

Keďže sa vyvíjaná aplikácia zaoberá sémantikou logiky prvého rádu, v tejto kapitole zhrnieme kľúčové pojmy z tejto oblasti. Vychádzame najmä z prednášok a poznámok k predmetu Logika pre informatikov [15], Švejdarovej učebnice [19] a prvej kapitoly knihy *Handbook of Mathematical Logic* [10].

1.1.1 Syntax

Logika prvého rádu je rodina formálnych jazykov, ktoré hovoria o objektoch a ich vlastnostiach. Symboly konkrétneho jazyka $\mathcal{L}$ logiky prvého rádu tvoria *individuové premenné* (napr. x, y, z) z nekonečnej spočítateľnej množiny $\mathcal{V}_{\mathcal{L}}$, *logické symboly* ($\neg, \vee, \wedge, \rightarrow, \doteq, \exists, \forall$), *pomocné symboly* (ľavá zátvorka, pravá zátvorka a čiarka) a *mimologické symboly*, tie sa ďalej delia na *individuové konštanty* zo spočítateľnej množiny $\mathcal{C}_{\mathcal{L}}$, *funkčné symboly* zo spočítateľnej množiny $\mathcal{F}_{\mathcal{L}}$ a *predikátové symboly* zo spočítateľnej množiny $\mathcal{P}_{\mathcal{L}}$. Individuové premenné, logické symboly a pomocné symboly sú rovnaké naprieč všetkými jazykmi logiky prvého rádu, čo však jednotlivé jazyky odlišuje sú použité mimologické symboly.

Predikátové symboly používame na označenie vlastností alebo vzťahov. V konkrétnom jazyku $\mathcal{L}$ vždy majú (spolu s funkčnými symbolmi) pevne zvolený počet argumentov, ktorý určuje ich *arita*, niekedy označovaná horným indexom. Predikátové symboly s aritou 1 (unárne) zvyknú označovať nejakú vlastnosť alebo stav (napr.

$zviaa^1, svieta^1$), zatiaľ čo predikátové symboly s aritou 2 a viac (n -árne pre $n \geq 2$) opisujú určitý vzťah svojich argumentov (napr. $nesie^2, medzi^3$).

Individuové konštanty odkazujú na konkrétny objekt. Často sa používajú na vyjadrenie mena alebo názvu objektu (napr. Alica, Bratislava). Konkrétny objekt môže byť označený viacerými alebo žiadnou individuovou konštantou, no každá individuová konštantá musí označovať práve jeden objekt.

Funkčné symboly používame na jednoznačné priradenie vstupných argumentov ku konkrétnemu objektu. Napríklad funkčný symbol $hlavne_mesto^1$ by mohol každému vstupnému objektu (štátu) priradovať ďalší objekt (jeho hlavné mesto). Nutnou podmienkou je, aby boli všade definované, teda aby každej n -tici vstupných argumentov priradovali konkrétny objekt, kde n je arita funkčného symbolu.

Zo symbolov jazyka vieme vytvárať postupnosti, pričom niektoré z nich označujeme ako *termy*. Medzi termy patria všetky individuové premenné a individuové konštanty. Okrem toho možno termy konštruovať aj pomocou funkčných symbolov a ich argumentov. Presnejšie, ak f predstavuje funkčný symbol s aritou n a $t_1, \dots, t_n$ sú termy, potom aj postupnosť symbolov $f(t_1, \dots, t_n)$ je term.

Najjednoduchšie tvrdenia v logike prvého rádu predstavujú *atomické formuly*. Medzi atomické formuly patria *rovnostné atómy*, teda postupnosti symbolov $t_1 \doteq t_2$, kde t_1 a t_2 sú termy, a *predikátové atómy*, teda postupnosti symbolov $P(t_1, \dots, t_n)$, kde P je predikátový symbol s aritou n a $t_1, \dots, t_n$ sú termy.

Nakoniec množinu všetkých *formúl* (tvrdení) jazyka $\mathcal{L}$ definujeme takto: Každá atomická formula je zároveň aj formula. Ak A je formula, tak aj postupnosť symbolov $\neg A$ je formula. Ak A aj B sú formuly, tak aj postupnosti symbolov $(A \wedge B)$, $(A \vee B)$, $(A \rightarrow B)$ sú formuly. A napokon ak x je individuová premenná a A je formula, tak aj postupnosti symbolov $\exists xA$ a $\forall xA$ sú formuly.

1.1.2 Sémantika

Prvým dôležitým pojmom týkajúcim sa sémantiky logiky prvého rádu je *štruktúra*, ktorá sa vždy viaže na nejaký konkrétny jazyk. Je ňou dvojica $\mathcal{M} = (D, i)$, kde D je ľubovoľnú neprázdna množina (doména) a i je zobrazenie (interpretačná funkcia). Úlohou interpretačnej funkcie je každej individuovej konštante priradiť prvok z D , každému funkčnému symbolu priradiť funkciu z D^n do D a každému predikátovému symbolu priradiť podmnožinu množiny D^n , kde n v oboch prípadoch označuje aritu daného symbolu. Štruktúry sú dôležitou súčasťou skúmania vlastností jazyka logiky prvého rádu, keďže poskytujú jeho presný matematický model.

Ako sme už ukázali, pomocou štruktúr vieme dať význam mimologickým symbolom. Premenným dávame význam prostredníctvom *ohodnotenia individuových premenných*, teda ľubovoľnej funkcie $e : \mathcal{V}_{\mathcal{L}} \rightarrow D$, ktorá každej premennej priraduje konkrétny prvok

domény. Potom *hodnota termu* t v štruktúre $\mathcal{M}$ pri ohodnotení premenných e je prvok z D označovaný $t^{\mathcal{M}}[e]$ definovaný nasledovne:

$$\begin{aligned} x^{\mathcal{M}}[e] &= e(x), \\ a^{\mathcal{M}}[e] &= i(a), \\ (f(t_1, \dots, t_n))^{\mathcal{M}}[e] &= i(f)(t_1^{\mathcal{M}}[e], \dots, t_n^{\mathcal{M}}[e]), \end{aligned}$$

pre všetky premenné x , všetky konštanty a , všetky funkčné symboly f s aritou n a všetky termy $t_1, \dots, t_n$.

Relácia $\mathcal{M} \models X[e]$ (čítame *štruktúra $\mathcal{M}$ spĺňa formulu X pri ohodnotení e*) je definovaná indukčne takto:

$$\begin{aligned} \mathcal{M} \models t_1 = t_2[e] &\text{ vtt } t_1^{\mathcal{M}}[e] = t_2^{\mathcal{M}}[e], \\ \mathcal{M} \models P(t_1, \dots, t_n)[e] &\text{ vtt } (t_1^{\mathcal{M}}[e], \dots, t_n^{\mathcal{M}}[e]) \in i(P), \\ \mathcal{M} \models \neg A[e] &\text{ vtt } \mathcal{M} \not\models A[e], \\ \mathcal{M} \models (A \wedge B)[e] &\text{ vtt } \mathcal{M} \models A[e] \text{ a zároveň } \mathcal{M} \models B[e], \\ \mathcal{M} \models (A \vee B)[e] &\text{ vtt } \mathcal{M} \models A[e] \text{ alebo } \mathcal{M} \models B[e], \\ \mathcal{M} \models (A \rightarrow B)[e] &\text{ vtt } \mathcal{M} \not\models A[e] \text{ alebo } \mathcal{M} \models B[e], \\ \mathcal{M} \models \exists x A[e] &\text{ vtt pre nejaký prvok } m \in D \text{ platí } \mathcal{M} \models A[e(x/m)], \\ \mathcal{M} \models \forall x A[e] &\text{ vtt pre každý prvok } m \in D \text{ platí } \mathcal{M} \models A[e(x/m)], \end{aligned}$$

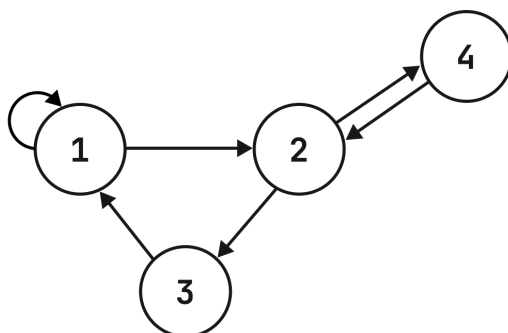
pre všetky premenné x , všetky formuly A, B , všetky termy $t_1, \dots, t_n$ a všetky predikátové symboly P s aritou n .

1.2 Grafové reprezentácie

V tejto podkapitole sú uvedené typy grafov použité v tejto práci spolu s krátkym vysvetlením možností ich použitia. Niektoré definície vychádzajú z Grimaldiho učebnice diskkrétnej matematiky [14].

1.2.1 Orientovaný graf

Orientovaný graf G je daný neprázdnu konečnou množinou vrcholov V a množinou usporiadaných dvojíc $E \subseteq V \times V$, resp. hrán. V diagramoch sú vrcholy väčšinou znázorňované ako kruhy, prípadne spolu s ich príslušným označením. Hrany sú zas vyobrazené pomocou šípok, ktorých smer určuje poradie vrcholov hrany (šípka vždy ukazuje na druhý vrchol v poradi). Grafy majú mnohoraké využitie pri modelovaní vzťahov, ktoré vyžadujú zachovať nejaký smer, postupnosť úkonov alebo vyjadriť určitú hierarchiu. Príklad bežného diagramu jednoduchého orientovaného grafu možno vidieť na obrázku 1.1, kde $V = \{1, 2, 3, 4\}$ a $E = \{(1, 1), (1, 2), (2, 3), (3, 1), (2, 4), (4, 2)\}$.



Obr. 1.1: Príklad vyobrazenia orientovaného grafu.

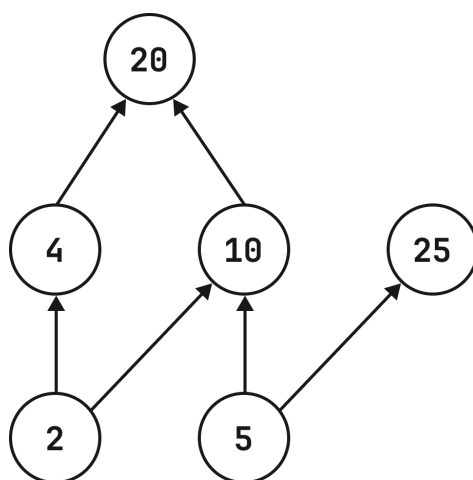
1.2.2 Hasseho diagram

Hasseho diagram je typ diagramu, ktorý je navrhnutý na zobrazenie čiastočných usporiadaní. Relácia $\leq$ na množine A sa nazýva *čiastočným usporiadaním*, ak je reflexívna, antisymetrická a tranzitívna. To znamená, že pre každé $x \in A$ musí platiť $x \leq x$ (prvok x je v relácii so sebou samým – reflexívna relácia), pre každé $x, y \in A$ platí $(x \leq y \wedge y \leq x) \Rightarrow x = y$ (ak je prvok x v relácii s prvkom y a y je v relácii s x , tak $x = y$ – antisymetrická relácia) a pre každé $x, y, z \in A$ platí $(x \leq y \wedge y \leq z) \Rightarrow x \leq z$ (ak je prvok x v relácii s prvkom y a y je v relácii s prvkom z , tak aj x je v relácii so z – tranzitívna relácia). Intuitívnejšie si ale tento pojem možno predstaviť ako dvojice prvkov, z ktorých prvý vždy predchádza druhému. Zvyčajne sa čiastočné usporiadanie označuje aj ako dvojica $(A, \leq)$.

Hasseho diagram slúži na prehľadnejšiu grafickú reprezentáciu čiastočných usporiadaní, keďže zachytáva iba bezprostredné vzťahy, bez zbytočného kreslenia hrán, ktoré vyplývajú z jeho vlastností. Pri jeho tvorbe však treba dbať na dodržiavanie určitých pravidiel:

- Každý prvok čiastočného usporiadania, teda prvok množiny A , je vrcholom v takomto grafe.
- Medzi ľubovoľnými prvkami $x, y \in A$ vytvárame hranu iba vtedy, keď $x \leq y$, $x \neq y$ a neexistuje taký prvok $z \in A$, rozdielny od x a y , pre ktorý platí $x \leq z \leq y$.
- Ak je medzi prvkami $x, y \in A$ hrana a v usporiadaní platí $x \leq y$, potom vrchol reprezentujúci prvok x kreslíme pod vrchol reprezentujúci prvok y .

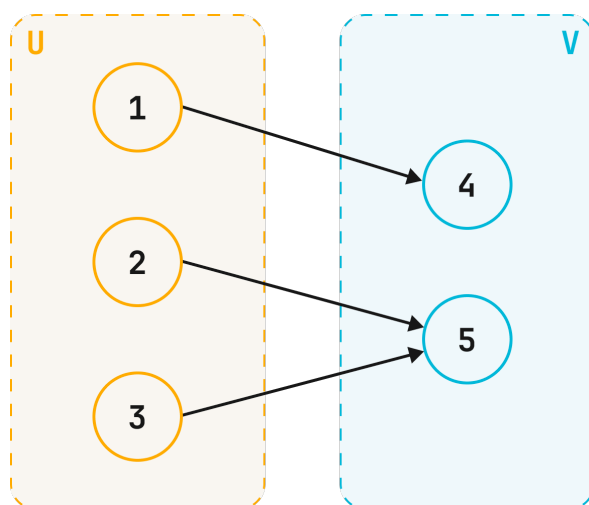
Na obrázku 1.2 možno vidieť príklad Hasseho diagramu pre čiastočné usporiadanie (A, R) , kde $A = \{2, 4, 5, 10, 20, 25\}$ a $x R y$, ak x delí y bezo zvyšku. Dobré je podotknúť, že aj prvky 2 a 5 sú v relácii s prvkom 20, no hrany vyjadrujúce tento vzťah, ako bolo vyššie spomínané, do diagramu explicitne nezakresľujeme.



Obr. 1.2: Príklad vyobrazenia Hasseho diagramu.

1.2.3 Bipartitný graf

Bipartitný graf je taký graf G , ktorého vrcholy možno rozdeliť do dvoch disjunktných podmnožín U a V tak, že $U \cap V = \emptyset$ a zároveň každá hrana spája jeden vrchol z U s ďalším vrcholom z V (alebo naopak). Sú nápomocné pri hľadaní a skúmaní vzťahov (relácií a zobrazení) medzi entitami dvoch rôznych tried, ako napríklad medzi študentmi a učiteľmi, autormi a publikáciami alebo receptami a ingredienciami. Pri menšom množstve dát a vhodnom rozložení vrcholov umožňujú prehľadnú vizualizáciu prítomných vzťahov. Ukážku diagramu konkrétného bipartitného grafu pre zobrazenie $f : \{1, 2, 3\} \rightarrow \{4, 5\}$, kde $f(1) = 4$, $f(2) = 5$ a $f(3) = 5$, so znázorneným rozdelením prvkov do množín U a V , ktoré si v tomto prípade možno predstaviť ako definičný obor a obor hodnôt zobrazenia, je vidieť na obrázku 1.3.



Obr. 1.3: Príklad vyobrazenia bipartitného grafu.

1.3 Použité technológie

V tejto podkapitole stručne predstavujeme kľúčové technológie použité pri implementácii.

1.3.1 React

React [16] je v súčasnosti jedna z najpopulárnejších JavaScriptových knižníc na tvorbu responzívnych webových aplikácií.

Základnú stavebnú jednotku tvorí komponent, čo je JavaScriptová funkcia, ktorá produkuje určitý strom elementov jazyka HTML, ďalej označovaný ako markup. Každý takýto komponent potom predstavuje konkrétnu časť používateľského rozhrania s vlastnou funkcionalitou a vzhľadom.

Vo väčšine prípadov sa markup komponentov zapisuje vo forme JavaScript XML (JSX), čo je syntaktické rozšírenie jazyka JavaScript. JSX je veľmi podobný značkovaciemu jazyku HTML, no umožňuje používanie vlastných značiek, ako aj jednoduchšiu manipuláciu s dynamickým obsahom. Toto je pre React kľúčové, keďže je tým umožnené skladať viaceré reaktívne komponenty do hierarchie, ktorá vytvára celú aplikáciu.

1.3.2 TypeScript

TypeScript [17] je programovací jazyk s podporou pre statické typy s konfigurovateľnou mierou typovej kontroly. Funguje ako syntaktické rozšírenie JavaScriptu, ktoré sa pri kompilácii naspäť zmení na čistý JavaScript. Práve táto vlastnosť zabezpečuje jeho širokú kompatibilitu s už existujúcim ekosystémom.

Statické typovanie pomáha odhaľovať chyby už v skorých fázach vývoja, zlepšuje čitateľnosť aj udržiateľnosť kódu a vývojárskym prostrediam umožňuje integráciu bohatšej funkcionality, ako napríklad pokročilejšie automatické dopĺňanie na základe kontextu alebo vyhľadávanie definícií. Svojimi prínosmi teda dokáže priamo aj nepriamo uľahčiť a spríjemniť prácu programátora.

Jazyk poskytuje možnosť definovať rozhrania pomocou kľúčového slova `interface`, ktoré upresňujú zamýšľaný tvar objektov. Kľúčové slovo `type` zas umožňuje vytváranie typových aliasov, či už primitívnych alebo komplexných typov. Okrem samotných typov podporuje typovú inferenciu, čo pomáha s prehľadnosťou programu, keďže často nie je potrebné typ premenných definovať manuálne.

1.3.3 React Flow

React Flow [2] je populárna knižnica na tvorbu grafových editorov, ktorá je súčasťou väčšieho projektu xyflow [5]. Zatiaľ čo React Flow je určený pre React, xyflow ponúka

plnú kompatibilitu aj s niektorými inými podobnými knižnicami. Jej veľkou výhodou je široká škála možností prispôsobenia jednotlivých aspektov grafu, ako aj ich jednoduchá implementácia.

Každý vrchol reprezentuje sada atribútov a príslušný React komponent. Atribúty musia byť minimálne tvorené pozíciou a unikátnym identifikátorom. Ďalej však môžeme pridávať vlastné atribúty, ako aj komponenty, ktoré sa dajú prispôsobiť špecifickým potrebám našej aplikácie. Na veľmi podobnom princípe sa dajú konfigurovať aj hrany grafu, tie sú tiež tvorené komponentmi a spolu s vrcholmi tvoria základnú štruktúru každého grafu.

Knižnica tiež poskytuje niektorú rozšírenú funkcionálnosť, akou je napríklad centrovanie grafov, uloženie a obnovenie stavu grafu alebo konfigurovateľný panel s nástrojmi.

1.3.4 Redux

Redux [4] je knižnica na správu stavu JavaScriptových aplikácií, často používaná spolu s knižnicou React na zjednodušenie tvorby komplexných používateľských rozhraní. *Stav aplikácie* je reprezentovaný iba ako obyčajný objekt (plain object), ktorý predstavuje centrálné miesto pre ukladanie a čítanie dát, čím výrazne uľahčuje ich zdieľanie naprieč komponentmi.

Stav v Reduxe nie je možné meniť priamo. Namiesto toho sa zmeny vykonávajú pomocou akcií a reducerov. *Akcie* sú objekty popisujúce konkrétnu udalosť v aplikácii. V prípade potreby môžu obsahovať aj doplňujúce údaje (payload). Tieto akcie ďalej spracúvajú *reducers*. To sú funkcie, ktoré na základe aktuálneho stavu a prijatej akcie určia nový stav aplikácie. Pre ich správne fungovanie je dôležité dbať na to, aby nemali vedľajšie efekty, teda aby reducer pre rovnaký stav a rovnakú akciu vždy vrátil rovnaký výsledok. Nemali by sa preto používať napríklad na generovanie náhodných hodnôt ani na komunikáciu s vonkajším svetom. Aplikácia môže obsahovať viacero reducerov, čo umožňuje rozdeliť logiku na menšie, nezávislé časti.

Na získavanie konkrétnych dát zo stavu sa používajú *selektory*. Ide o funkcie, ktoré na základe súčasného globálneho stavu vyberajú alebo odvodzujú potrebné údaje, čím zjednodušujú a sprehľadňujú prácu s dátami.

Kapitola 2

Súvisiace práce

V tejto kapitole sa podrobnejšie venujeme aplikáciám Prieskumník štruktúr a Prieskumník grafových štruktúr, ktoré sú pre túto prácu kľúčové. Postupne vysvetlíme, aké problémy sa snažia vyriešiť, ako fungujú a aké sú ich nedostatky. V závere kapitoly stručne predstavíme aplikáciu Logický pracovný zošit, do ktorej sú spomínané aplikácie integrované.

2.1 Prieskumník štruktúr

Prieskumník štruktúr je jednou z webových aplikácií používaných na predmete Logika pre informatikov. Jej primárnou úlohou je umožniť študentom interaktívne vytvárať vlastný jazyk a štruktúru logiky prvého rádu. V takto definovanej štruktúre je ďalej možná tvorba formúl a analýza ich pravdivosti s rýchlou spätnou väzbou. Jej návrh bol inšpirovaný desktopovou aplikáciou Tarski's World [8] používanou v učebnici *Language, Proof, and Logic* [9]. Táto aplikácia tiež umožňuje skúmať pravdivosť formúl, avšak iba v špeciálnych, menej všeobecných svetoch (predstavujúcich objekty na šachovnici) a bez možnosti meniť preddefinovaný jazyk.

Pôvodná aplikácia vznikla v rámci viacerých bakalárskych prác. Konkrétne ide o prácu Milana Cifru [12], ktorá sa zaoberá samotným vývojom Prieskumníka štruktúr, prácu Richarda Tótha [21], rozširujúcu prieskumník o Henkinovu-Hintikkovu hru a prácu Miroslava Balucha [7], ktorá priniesla alternatívny grafový pohľad na štruktúru. Túto poslednú prácu ešte bližšie rozoberieme v nasledujúcej podkapitole 2.2.

Implementácia aplikácie sa však časom stala neaktuálnou a nejednotnou, keďže bola výsledkom viacerých bakalárskych prác využívajúcich rôzne prístupy k použitým technológiám. To spôsobilo jej ťažkú udržiateľnosť, ako aj náročnosť rozširovania o ďalšiu funkcionálnosť.

Cieľom bakalárskej práce Jozefa Filipa [13] bolo vytvoriť novú, jednotnú verziu pôvodnej aplikácie bez grafového pohľadu a s novým spôsobom využitia Henkinovej-

Obr. 2.1: Používateľské rozhranie reimplentovanej verzie Prieskumníka štruktúr.

Hintikkovej hry. Túto verziu možno vidieť na obrázku 2.1.

V bloku Language v ľavej hornej časti obrázka sa nachádzajú textové vstupy na definovanie mimologických symbolov jazyka. V prípade predikátových a funkčných symbolov sa definuje aj ich arita vo formáte *symbol/arita*. Pod definíciou jazyka logiky prvého rádu sa v bloku Structure definuje samotná štruktúra, čo vyžaduje zadanie domény a určenie interpretácie jednotlivých mimologických symbolov. Aplikácia poskytuje iba jeden, tzv. množinový pohľad na interpretáciu predikátových a funkčných symbolov, ktorý je opäť realizovaný pomocou textového vstupu.

V bloku Truth of formulas v pravej časti aplikácie používateľ definuje formuly. Po vytvorení validného jazyka a štruktúry môže skúmať pravdivosť týchto formúl v danej štruktúre. Najprv však musí sám odhadnúť a zvoliť, či je formula pravdivá, alebo nie. Svoj odhad si následne overí prostredníctvom Henkinovej-Hintikkovej hry. Tá funguje na princípe dialógu, v ktorom používateľ postupne dostáva otázky o pravdivosti čoraz menších podformúl pôvodnej formuly. Keď je už podformula atomická, hra používateľovi odhalí správnosť jeho predpokladu spolu s vysvetlením, prečo bol alebo nebol správny. Príklad dialógu je zobrazený na obrázku 2.2.

Asi najväčším problémom novej implementácie v porovnaní s pôvodnou je nedostatok spôsobov vizualizácie a tvorby interpretácií predikátových a funkčných symbolov.

You assume that $\mathcal{M} \models (figúrka(x) \rightarrow \neg poličko(x)) [e(x/\oplus)]$

Which option is true?

$\mathcal{M} \models figúrka(x) [e(x/\oplus)]$

$\mathcal{M} \models \neg poličko(x) [e(x/\oplus)]$

[Change](#) $\mathcal{M} \models \neg poličko(x) [e(x/\oplus)]$

You assume that $\mathcal{M} \models \neg poličko(x) [e(x/\oplus)]$

Then $\mathcal{M} \models poličko(x) [e(x/\oplus)]$

[Continue](#)

You assume that $\mathcal{M} \models poličko(x) [e(x/\oplus)]$

You win, $\mathcal{M} \models poličko(x) [e(x/\oplus)]$, since $(\oplus) \notin i(poličko)$

Your initial assumption that $\mathcal{M} \models (\forall x (figúrka(x) \rightarrow \neg poličko(x)) \wedge \forall x (poličko(x) \rightarrow \neg \exists y farba(y) = x)) [e]$ was correct.

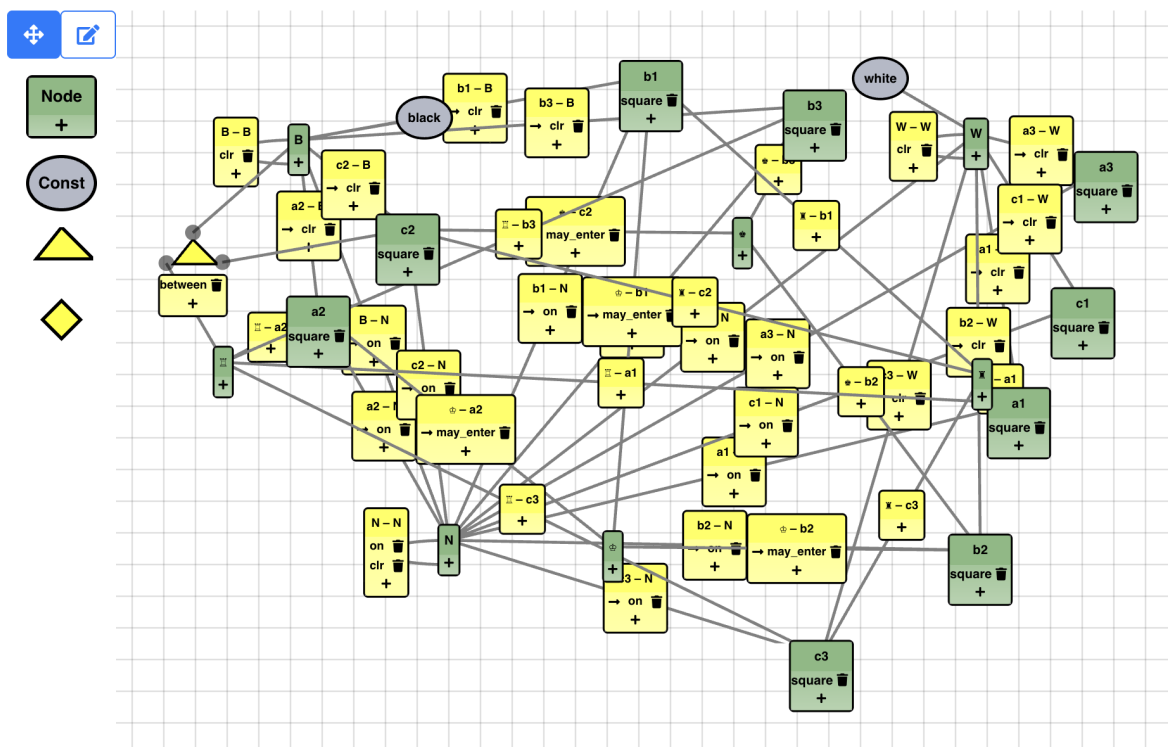
Obr. 2.2: Príklad dialógu Henkinovej-Hintikkovej hry.

Množinová, resp. textová reprezentácia sa už pri pomerne malých interpretáciách stáva neprehľadnou a ťažko editovateľnou, keďže vyžaduje dodržiavanie správnej syntaxe pri zápise. Predošlá verzia tento problém čiastočne riešila pridaním dvoch ďalších alternatívnych pohľadov. Konkrétne išlo o tzv. maticový a databázový pohľad, v ktorých bolo možné pomocou zaškrtnutia alebo výberu zo zoznamu určiť, ktoré prvky sa majú v interpretácii vyskytnúť.

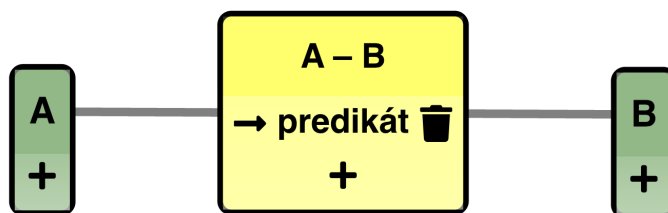
2.2 Prieskumník grafových štruktúr

Úlohou bakalárskej práce Miroslava Balucha [7] bolo rozšíriť pôvodnú aplikáciu Prieskumník štruktúr o nový pohľad na štruktúry logiky prvého rádu vo forme grafového editora. V jednom grafe sa mu podarilo pomocou rôznych typov vrcholov znázorniť konštanty, prvky domény a pomocou hrán aj vzťahy medzi nimi. Príklad zobrazenia konkrétnej štruktúry je na obrázku 2.3.

S grafom možno interagovať v dvoch režimoch. Prvým je pohybový režim, ktorý umožňuje iba mazanie a zmenu pozície jednotlivých objektov. V druhom, editovacom režime sú už sprístupnené funkcie ako vytváranie predikátových a funkčných symbolov, tvorba nových vzťahov alebo premenovávanie prvkov domény. Tlačidlá, pomocou ktorých sa medzi režimami prepína, sú viditeľné v ľavej hornej časti obrázka. Pod týmito tlačidlami sú umiestnené jednotlivé typy vrcholov. Ich potiahnutím do priestoru grafu sa vytvorí nová inštancia príslušného typu. Zelené vrcholy reprezentujú prvky domény a sivé konštanty. Žlté vrcholy pomáhajú pri tvorbe ternárnych a kvaternárnych vzťahov.



Obr. 2.3: Grafový editor v predošlej implementácii Prieskumníka štruktúr



Obr. 2.4: Príklad znázornenia binárneho vzťahu.

Binárny vzťah medzi prvkami domény je znázornený pomocou hrany spolu s jej popisom (viď obrázok 2.4). Do popisu možno pridávať alebo odoberať predikátové a funkčné symboly, ktoré majú tento vzťah obsahovať vo svojej interpretácii. Pomocou šípky pri názve predikátu sa určuje smer vzťahu. Pri ternárnych a kvaternárnych vzťahoch je nutné najprv spojiť všetky prvky domény s príslušným vrcholom, pri ktorom sa následne dá meniť popis už spomínaným spôsobom.

Ako sa však časom ukázalo, implementácia mala niekoľko zásadných problémov. Hlavným z nich bola snaha nahradiť takmer celú funkcionalitu Prieskumníka štruktúr, čo na viacerých miestach viedlo ku kompromisom znižujúcim prehľadnosť aj intuitívnosť rozhrania. Azda najväčším vinníkom zlej prehľadnosti, resp. čitateľnosti, je spôsob zobrazovania binárnych vzťahov. Tých sa v štruktúrach používaných na predmete vyskytuje pomerne veľa, čo v kombinácii s ich rozsiahlou vizualizáciou vedie k prekryvaniu ostatných prvkov a výraznému zníženiu schopnosti orientácie sa v zobrazení.

Návrh niektorých ovládacích prvkov tiež pôsobil vo výsledku ťažkopádne, ako napríklad oddelenie editovania a presúvania do dvoch samostatných režimov. Ďalším problémom je nedostatok zaujímavých a unikátnych funkcií v porovnaní s textovými vstupmi, ktoré by ho spravili pre študentov atraktívnejším. Spomínané nedostatky sa prejavili aj na jeho nízkej popularite na predmete, kde sa prakticky vôbec nevyužíval.

2.3 Logický pracovný zošit

Prieskumník štruktúr a ďalšie nástroje na výučbu matematickej logiky na predmete boli v čase svojho vzniku dostupné iba v podobe samostatných aplikácií. S ich rastúcim počtom sa však používanie viacerých rôznych aplikácií pri riešení zadaní stávalo nepraktickým. Tento problém vyriešila webová aplikácia Logický pracovný zošit, ktorá spája všetky používané nástroje do jedného celku. Učiteľom predmetu tým umožňuje vytvárať ucelenejšie cvičenia a žiakom uľahčuje ich vypracovávanie, ako aj odovzdávanie. Ďalšou výhodou pracovného zošita je priebežné ukladanie práce žiakov, vďaka čomu sa k nej môžu kedykoľvek vrátiť. Učitelia majú zároveň k uloženým prácam prístup, čo im dovoľuje ich prezerať a pridávať komentáre. Vývojom aplikácie sa vo svojej bakalárskej práci zaoberal Matej Mok [18].

Kapitola 3

Návrh

V tejto kapitole najprv zdefinujeme požiadavky kladené na výslednú aplikáciu. Následne predstavíme návrh jednotlivých riešení spolu s vysvetlením, ako sme k nim dospeli. V závere sa zameriame na návrh stavu nových častí aplikácie a jeho zakomponovanie do už existujúceho riešenia.

3.1 Požiadavky

Požiadavky na rozšírenie aplikácie sa postupne vyvíjali a rozširovali, čo viedlo k pridaniu pomerne veľkého množstva novej funkcionality nad rámec grafových editorov. V tejto podkapitole opíšeme všetky požiadavky a vysvetlíme dôvod ich zaradenia.

3.1.1 Zámer rozšírenia aplikácie

Hlavným cieľom tejto práce je zlepšiť použiteľnosť existujúcej aplikácie Prieskumník štruktúr, používanej na predmete Logika pre informatikov. Ako bolo už bližšie spomenuté v podkapitole 2.1 venovanej práve tomuto nástroju, reimplementovaná verzia poskytuje iba veľmi obmedzený spôsob vizualizácie a manipulácie s interpretáciami predikátových a funkčných symbolov.

Primárnym zlepšením v tejto oblasti má byť rozšírenie nástroja o grafové editory, ktoré používateľom pomôžu pri vizualizácii a vytváraní binárnych predikátov a unárnych funkcií logiky prvého rádu. Konkrétne sa jedná o orientovaný graf, bipartitný graf a Hasseho diagram. Každý z nich bol zvolený pre svoje unikátne vlastnosti, ktoré môžu byť nápomocné v rozličných situáciach.

Aplikáciu by sme však radi posunuli ďalej aj v iných smeroch, či už integráciou ďalších editorov, pridaním novej funkcionality alebo vylepšením tej existujúcej.

3.1.2 Požiadavky na grafové editory

Každý z grafov by mal z hľadiska funkčnosti slúžiť ako plnohodnotná alternatíva k už existujúcemu množinovému pohľadu. Musí teda zobrazovať jednotlivé usporiadané dvojice, resp. vzťahy, ktoré konkrétna interpretácia opisuje a prvky domény, medzi ktorými tieto vzťahy vznikajú. Vzťahy sa tiež musia dať vytvárať a mazať.

Vyobrazenie grafov by malo byť prispôsobené konkrétnym vlastnostiam každej reprezentácie. Bipartitný graf musí jasne znázorňovať rozdelenie prvkov do dvoch skupín, pričom vytváranie nových hrán je dovolené iba z jednej z nich. Hasseho diagram by zas mal poskytnúť možnosť automatického vytvorenia hierarchie typickej pre túto reprezentáciu. Tiež by sa malo dbať na to, aby používateľ mohol vytvárať iba vzťahy spĺňajúce podmienky čiastočného usporiadania (bližšie opísané v sekcii 1.2.2).

Oproti množinovému pohľadu navyše pribudnú vylepšenia zamerané na prehľadnejšie zobrazenie ďalších vzťahov v štruktúre. Konkrétne by mali byť nejakým spôsobom vizualizované unárne predikáty, keďže nesú informáciu o vlastnostiach jednotlivých prvkov. Všetky unárne predikáty sa však vždy nemusia zobrazovať. Používateľ si z nich bude môcť vybrať a na základe príslušnosti prvkov domény k ich interpretáciám bude možné tieto prvky filtrovať, pričom prvky nevyskytujúce sa v žiadnej z vybraných interpretácií sa v grafe nezobrazia. Okrem toho by sa mali vhodným spôsobom zobrazovať aj konštanty, ktorými sú jednotlivé prvky domény označené.

3.1.3 Ďalšie požiadavky

Počas vývoja aplikácie sme sa rozhodli popri grafových editoroch pridať aj ďalšiu prínosnú funkcionálnu. Požiadavky na tieto rozšírenia ďalej v krátkosti popisujeme.

Maticový a databázový editor: Jednou z pridaných požiadaviek je reimplemen-tácia maticového a databázového editora z pôvodnej verzie Prieskumníka štruktúr, ktoré vznikli v rámci práce Milana Cifru [12]. Ich účel je podobný ako pri grafových reprezentáciách, no okrem poskytnutia alternatívneho pohľadu dokážu reprezentovať aj iné než iba binárne predikáty a unárne funkcie. Pri týchto editoroch sa taktiež vyžaduje možnosť zobrazovať unárne predikáty a využívať ich na filtrovanie.

Editor interpretácií funkcií rozborom prípadov: Ďalšie vylepšenie má za cieľ uľahčiť tvorbu interpretácií funkčných symbolov. To býva často zdĺhavé, keďže doposiaľ spomínané riešenia vyžadujú explicitné definovanie všetkých prípadov. Dá sa to však vylepšiť nástrojom, ktorý umožní definovať podmienky na argumenty funkcie spolu s príslušnými hodnotami, ktoré má funkcia pri ich splnení nadobúdať. Požiadavkou je preto vytvorenie nástroja založeného na takomto princípe.

Dopyty: Toto rozšírenie na rozdiel od ostatných neslúži na tvorbu interpretácií. Malo by predstavovať samostatnú sekciu aplikácie určenú na definovanie otvorených formúl a následné získanie všetkých ohodnotení, pri ktorých sú tieto formuly splnené.



Obr. 3.1: Návrh spoločného rozhrania editorov.

Výsledok takéhoto dopytu by sa mal vhodne zobrazit' v podobe tabuľky.

RefaktORIZÁCIA ČASTÍ APLIKÁCIE: Aplikácia si v aktuálnej verzii ukladá jednotlivé časti stavu primárne v textovej podobe, keďže to bol doteraz jediný spôsob reprezentácie dát. Pri našich nových požiadavkách je to však obmedzujúce a bude potrebné dáta ukladať štruktúrovanejším spôsobom. Taktiež logika zodpovedná za generovanie Henkinovej-Hintikkovej hry by mala v určitých situáciách náhodne vyberať podformulu alebo prvok domény, no v súčasnej verzii tomu tak nie je. Požaduje sa preto oprava aj tohto správania.

3.2 Spoločné rozhranie editorov

Na obrázku 3.1 možno vidieť návrh rozhrania, do ktorého sú jednotlivé editory vkladané. Jeho cieľom je poskytnúť konzistentné rozhranie spoločných ovládacích prvkov naprieč všetkými druhmi editorov. Dokopy sa skladá z nasledujúcich komponentov:

Hlavička editora: V ľavej časti hlavičky je zobrazený predikátový alebo funkčný symbol spolu s názvom aktuálneho editora. V pravej časti sa nachádza tlačidlo, po ktorého stlačení sa zobrazí zoznam všetkých kompatibilných editorov. Výber konkrétneho editora z ponuky zobrazenie automaticky prepne.

Panel unárnych predikátov: Tento komponent tvorí najdôležitejšiu časť spoločného rozhrania. Ústredným prvkom je horizontálny zoznam zaškrŕavacích tlačidiel reprezentujúcich jednotlivé unárne predikáty, pričom každému z nich je priradená vlastná farba. Ako aj neskôr v návrhu grafových editorov 3.3 a tabuľkových editorov 3.4 uvidíme, kompatibilné editory túto farbu rôzne využívajú pri ich vizualizácii. Tlačidlo s dvoma fajkami, umiestnené naľavo, slúži na hromadné zaškrŕnutie všetkých unárnych predikátov. Úplne vľavo sa nachádza zaškrŕvacie tlačidlo označené písmenom

„D“, tzv. doménové, ktorého funkcia je podrobnejšie vysvetlená v časti 3.2.1 venovanej filtrovaniu.

Filter prvkov domény: Tlačidlo na rozbalenie filtrov je umiestnené napravo od panela unárnych predikátov. Po jeho stlačení sa zobrazí komponent (na obrázku úplne vpravo) so zaškrtávacími tlačidlami reprezentujúcimi prvky domény. Pri každom prvku sú zároveň vo forme krúžkov zobrazené farby unárnych predikátov, do ktorých interpretácií patrí. Odškrtnutím konkrétneho prvku indikujeme, že sa nemá nachádzať v zobrazení editora.

Chybový panel: Tento komponent sa zobrazí iba v prípade, že v interpretácii nastala chyba. V ľavej časti je vypísaná chybová správa. Ak sa navyše jedná o chybu, ktorú je možné automaticky opraviť, na pravej strane zároveň pribudne tlačidlo na jej opravu.

Zobrazenie editora: Do tejto časti je vložený komponent zvoleného editora.

3.2.1 Filtrovanie

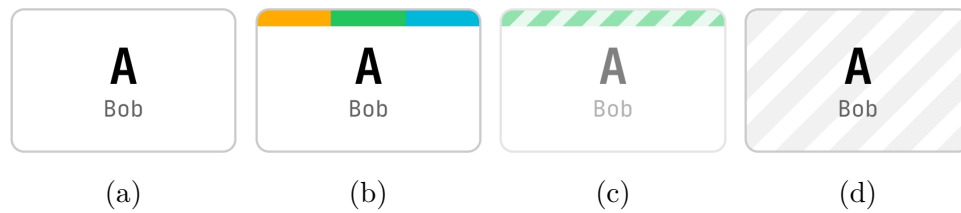
Každý editor podporujúci filtrovanie nejakým spôsobom zobrazuje prvky domény. To, aké prvky budú zobrazené, ovplyvňuje viacero faktorov, ktoré sme počas vývoja menili.

V jednom z prvých návrhov sa vždy zobrazovala celá doména. Neskôr sme toto správanie zmenili tak, aby boli zobrazené iba tie prvky, ktoré sa zároveň nachádzajú v interpretáciách zvolených unárnych predikátov. Videli sme však potenciálne využitie v oboch prístupoch. Zobrazenie celej domény je prínosné najmä v prípadoch, keď sa niektoré prvky domény nenachádzajú v interpretácii žiadneho unárneho predikátu, no používateľ by ich aj napriek tomu chcel v zobrazení ponechať. Filtrovanie prvkov na základe zvolených interpretácií unárnych predikátov zas umožňuje pohľad sprehľadniť odstránením nezaujímavých prvkov. Preto sme sa rozhodli oba prístupy skombinovať pridaním tlačidla domény. Po jeho zaškrtnutí sa vždy zobrazuje celá doména, v opačnom prípade sa zobrazenie riadi interpretáciami unárnych predikátov.

Komponent filtra prvkov domény je separátny. Ideou za ním je umožniť selektívnejšie filtrovanie. Výsledná zobrazená doména predstavuje prienik prvkov vybraných pomocou unárnych predikátov a prvkov zvolených vo filtri domény.

3.3 Grafové editory

Táto podkapitola sa zameriava na návrh editorov pre tri zvolené grafové reprezentácie, konkrétne ide o orientovaný graf, bipartitný graf a Hasseho diagram (bližšie opísané v podkapitole 1.2). Súčasťou je aj návrh jednotlivých komponentov grafu a spôsobu akým s nimi bude používateľ interagovať.



Obr. 3.2: Návrh vizuálu vrcholov.

3.3.1 Orientovaný graf

Orientovaný graf predstavuje v našom rozšírení základnú grafovú reprezentáciu, keďže jeho jednotlivé komponenty, ako vrcholy a hrany, možno využiť aj v ostatných grafových reprezentáciách. Pri jeho návrhu sme sa preto predovšetkým zamerali na prehľadnú vizualizáciu všetkých požadovaných informácií a intuitívne ovládanie dostupných funkcií.

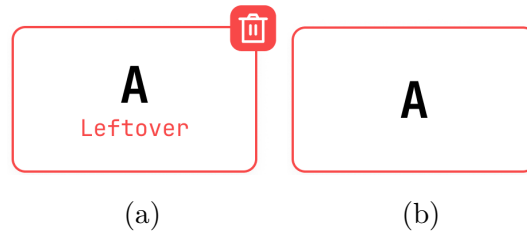
V grafe prvky domény prirodzene predstavujú vrcholy, zatiaľ čo vzťahy sú reprezentované pomocou hrán. Konštanty a unárne predikáty som sa taktiež rozhodol zakomponovať ako súčasť vrcholov, keďže ide o informácie prislúchajúce prvkom domény. Výzvou, ktorú takéto riešenie však prináša, je zachovať rovnováhu medzi množstvom informácií obsiahnutých vo vrcholoch a ich prehľadnosťou.

Prístup, ktorý som pri návrhu vrcholov zvolil, možno vidieť na obrázku 3.2. Najväčší typografický dôraz sa kladie na samotný prvok domény (na obrázku veľké písmeno A). Pod ním sa nachádzajú konštanty reprezentujúce tento prvok (na obrázku nápis Bob). Na druhom vrchole zľava (b) možno vidieť lištu predstavujúcu unárne predikáty, do interpretácie ktorých daný prvok patrí, pričom každá farba symbolizuje konkrétny predikát. Práve použitie farieb namiesto iných alternatív, ako napríklad nápisov s názvom unárneho predikátu, má pomôcť s celkovou prehľadnosťou vrcholov.

Pri takomto riešení sme však narazili na problém s prístupnosťou. Môj pôvodný výber totiž obsahoval aj také kombinácie farieb, ktoré by pre ľudí s poruchami farebného videnia neboli dostatočne rozlíšiteľné. Pri výbere vhodnej palety aj pre takéto prípady sme sa inšpirovali článkom *Qualitative Color Schemes* [20]. Ten obsahuje niekoľko príkladov vhodných farebných paliet spolu s návodom, ako s nimi zaobchádzať.

Posledné dva varianty zobrazenia vrcholov na obrázku 3.2 sa aktivujú iba v prípade, keď používateľ prejde kurzorom na zaškrŕavacie tlačidlo unárneho predikátu. Ak by zaškrŕnutie tohto tlačidla znamenalo pridanie vrcholu do zobrazenia, objaví sa jeho čiastočne priehľadná verzia (c). Ak by naopak zaškrŕnutie viedlo k skrytiu daného vrcholu, je nahradený jeho vyšrafovanou verziou (d). Cieľom je používateľovi jasnejšie naznačiť, aký vplyv bude mať jeho voľba na výsledné zobrazenie grafu.

Ďalej sme sa rozhodli pomocou vrcholov vyjadriť aj určité chybové stavy. Tie môžu nastať v dvoch prípadoch. Jedným z nich je situácia, keď interpretácia obsahuje prvok, ktorý nepatrí do domény. Vrchol reprezentujúci takýto prvok je zobrazený na obrázku



Obr. 3.3: Návrh vizuálu chybových vrcholov.

3.3(a). Okrem vizuálnej informácie v podobe červeného orámovania a nápisu poskytuje aj možnosť chybný prvok odstrániť kliknutím na tlačidlo koša, čím sa interpretácia opraví. Druhý prípad nastáva pri interpretáciách funkčných symbolov. Vtedy je vrcholom na obrázku 3.3(b) reprezentovaný prvok, ktorý nemá priradenú hodnotu alebo ich má priradených viac.

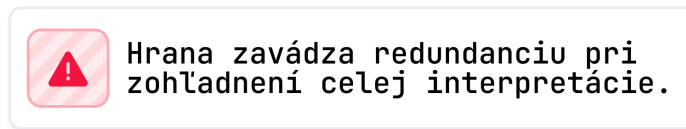
Dôležitou funkcionalitou grafu je aj spôsob vytvárania hrán. Na rozdiel od predošlej implementácie grafového zobrazenia (2.2) sme sa rozhodli nerozdeľovať ovládanie na dva režimy (pohybový a editovací). To si však vyžaduje, aby bola do vrcholu naraz zakomponovaná možnosť jeho presúvania, ako aj tvorby novej hrany. Žiadne z riešení ponúkaných knižnicou nám nevyhovovalo, preto sme sa rozhodli pre vlastný prístup, pri ktorom je vrchol rozdelený na dve časti – jednu určenú na vytváranie hrán a druhú na jeho presúvanie. Vytváranie hrany iniciujeme umiestnením kurzora nad určitú oblasť po obvodě vrcholu. Používateľovi je táto oblasť naznačená jej stmavením a zmenou typu kurzora. Následným potiahnutím z tejto oblasti nad iný vrchol a pustením sa proces vytvárania ukončí. Používateľ sa však môže pokúsiť vytvoriť aj neplatnú hranu, ako napríklad duplicitnú. Počas vytvárania sa preto hrana zobrazuje červenou alebo zelenou farbou podľa toho, či je jej vytvorenie validné. Časť vrcholu určená na posúvanie sa nachádza na zvyšnej ploche, teda okolo jeho stredu.

3.3.2 Hasseho diagram

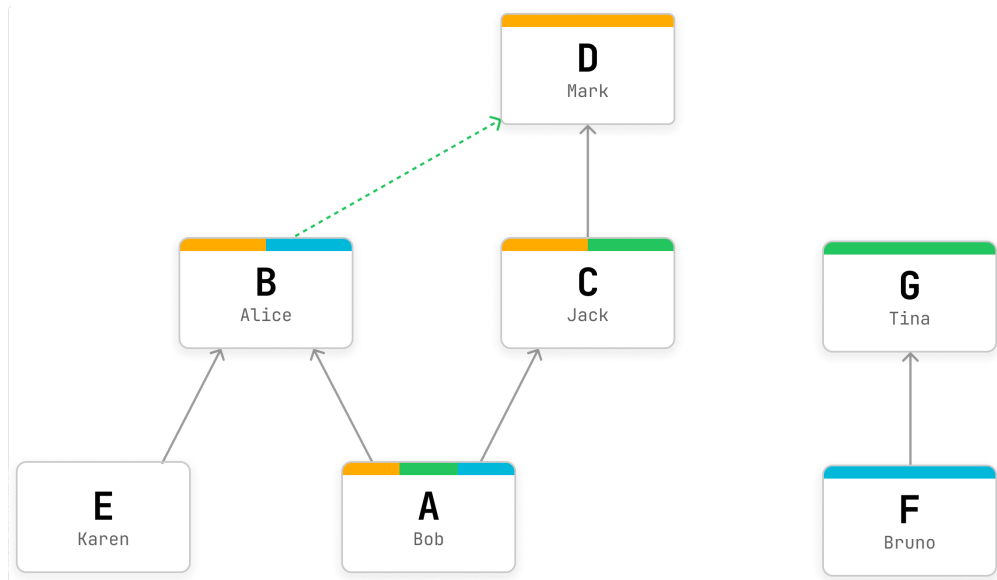
V editore Hasseho diagramu zostal spôsob fungovania vrcholov a hrán oproti orientovanému grafu v zásade nezmenený. Zamerali sme sa najmä na overovanie, či interpretácia binárneho predikátu skutočne tvorí čiastočné usporiadanie, keďže ide o nevyhnutný predpoklad pre zmyslupnosť tejto reprezentácie.

Pri vytváraní nových hrán sa preto kontroluje, či by ich pridaním ostali zachované všetky podmienky čiastočného usporiadania. Ak by tomu tak nebolo, používateľ je už počas vytvárania hrany upozornený varovnou správou v spodnej časti rozhrania grafu s vysvetlením, akú vlastnosť by ňou porušil. Príklad takejto správy možno vidieť na obrázku 3.4.

Ďalšou požiadavkou bolo automatické usporiadanie vrcholov do hierarchickej štruk-



Obr. 3.4: Príklad varovnej správy Hasseho diagramu.



Obr. 3.5: Návrh vizuálu pre Hasseho diagram.

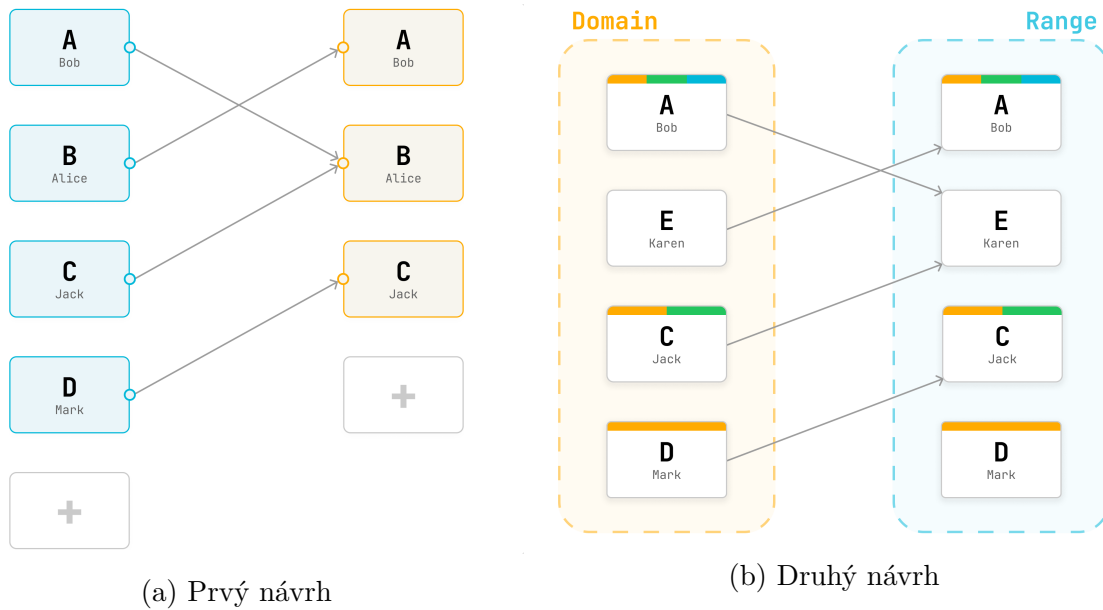
túry typickej pre túto reprezentáciu. Rozhodli sme sa však, že nebudeme používateľa akokoľvek obmedzovať za účelom dodržiavania tejto hierarchie. Namiesto toho mu poskytneme dobrý základ rozmiestnenia vrcholov, ktorý si môže ďalej prispôbiť podľa vlastných predstáv. Samotné rozloženie preto prebieha iba pri prvom otvorení grafu alebo na vyžiadanie používateľa stlačením tlačidla na paneli nástrojov.

Môj návrh je zobrazený na obrázku 3.5. Viditeľná je jasná hierarchia vrcholov s orientovanými hranami smerujúcimi nahor. Podobne ako v orientovanom grafe je používateľovi zelenou prerušovanou čiarou naznačená korektnosť hrany, ktorú sa práve snaží pridať (tu je hrana v poriadku). V opačnom prípade by bola hrana zvýraznená červenou farbou a používateľovi by sa zobrazila už spomínaná varovná správa.

3.3.3 Bipartitný graf

Špecifickým pre bipartitný graf je, že na rozdiel od orientovaného grafu a Hasseho diagramu každému prvku domény vždy prislúchajú dva vrcholy. Po jednom sa nachádzajú v skupine reprezentujúcej obor a koobor binárnej relácie alebo zobrazenia interpretujúceho predikátový, resp. funkčný symbol. Je to tak kvôli tomu, že obor aj koobor v prípade prvorádových štruktúr predstavuje doména štruktúry.

Prvý návrh editora bipartitného grafu možno vidieť na obrázku 3.6(a). Hrany re-



Obr. 3.6: Návrhy vizuálu pre bipartitný graf.

prezentujú šípky smerujúce zľava doprava, pričom ide o jediný smer, v ktorom sa dajú vytvárať. Vrcholy sú rozdelené do dvoch stĺpcov, čo je naznačené aj ich rozdielnou farbou. Každý stĺpec reprezentuje ľubovoľnú podmnožinu prvkov danej domény. Používateľovi je tiež umožnené meniť poradie vrcholov, ale iba v rámci príslušnej skupiny, presúvanie medzi skupinami nie je dovolené. Posledný, menej výrazný vrchol sa líši od ostatných, pretože slúži ako tlačidlo na pridávanie ďalších prvkov do príslušného stĺpca. Vytváranie hrán sa realizuje potiahnutím z krúžku počiatočného na krúžok koncového vrcholu.

Na uvedenom riešení nám však nevyhovoval odlišný vizuál vrcholov v porovnaní s ostatnými grafovými editormi. Navyše, možnosť samostatného výberu vrcholov oboch skupín sme v kombinácii s existujúcimi možnosťami filtrovania nepovažovali za vhodnú, keďže pre používateľa by mohlo byť náročné pochopiť, ako jednotlivé filtre ovplyvnili výsledné zobrazenie. Rozhodli sme sa preto náš návrh ešte upraviť a dopracovať do druhej, finálnej verzie, ktorá je vyobrazená na obrázku 3.6(b).

Jednou zo zmien je zjednotenie funkcionality a vizuálu vrcholov podľa vzoru orientovaného grafu. Príslušnosť vrcholov do skupiny je naznačená pomocou dvoch kontajnerov s rozdielnou farbou a názvom danej skupiny. Tiež sme odstránili tlačidlá na pridávanie ďalších prvkov do skupiny. V dôsledku toho obe časti vždy obsahujú rovnakú podmnožinu domény, ovládanú iba pomocou filtrov. Týmto úpravami sa nám podarilo dosiahnuť konzistentnejšie správanie naprieč grafovými editormi. Všetky ostatné funkcie sme ponechali.

3.4 Tabuľkové editory

V tejto podkapitole opíšeme návrh ďalších dvoch typov editorov založených na tabuľkovom zobrazení. Oba sú unikátne tým, že svoju podobu prispôsobujú jednak tomu, či reprezentujú interpretáciu predikátového alebo funkčného symbolu, ako aj arite daného symbolu.

3.4.1 Maticový editor

Návrh maticového editora reprezentujúceho konkrétny binárny predikát je znázornený na obrázku 3.7. Horizontálnu aj vertikálnu hlavičku tabuľky tvoria prvky domény, v tomto prípade sú to veľké písmená. Príslušnosť prvkov k interpretáciám unárnych predikátov je vizualizovaná podobne ako pri grafoch, teda tiež pomocou farieb. Pre nedostatok priestoru bolo však v tomto prípade potrebné zvoliť kompaktnejšie riešenie v podobe krúžkov.

Každá usporiadaná dvojica prvkov domény je reprezentovaná zaškrtávacím políčkou, ktorým je možné jednoducho zaznačiť, či patrí alebo nepatrí do interpretácie. Editor tiež podporuje zobrazenie interpretácií unárnych predikátov. V takom prípade sa zobrazí iba horizontálna hlavička, keďže je potrebné zaškrtávať konkrétny prvok.

Pri editovaní interpretácií unárnych a binárnych funkčných symbolov sú zaškrtávacie políčka nahradené textovými vstupmi. Riadky a stĺpce tabuľky zodpovedajú argumentom funkcie, pričom každý textový vstup predstavuje hodnotu, ktorú má unárna alebo binárna funkcia pre príslušný argument, resp. usporiadanú dvojicu argumentov nadobúdať. Inak je jeho rozloženie v zásade identické. Obsah týchto textových vstupov je potrebné aj validovať, aby sa overilo, že používateľ skutočne zadáva výhradne prvky domény. V prípade chyby sa príslušné pole zvýrazní červenou farbou a používateľovi sa na chybovom paneli zobrazí vysvetlenie problému.

Doména	A 	B 	C 	D 	E
A 	☒	☒	☐	☒	☐
B 	☐	☐	☐	☐	☒
C 	☒	☒	☐	☐	☐
D 	☐	☐	☐	☒	☐
E	☐	☒	☒	☒	☐

Obr. 3.7: Návrh rozhrania pre maticový editor.

3.4.2 Databázový editor

Príklad databázového editora interpretácie konkrétneho ternárneho predikátu možno vidieť na obrázku 3.8. V tejto reprezentácii sú jednotlivé trojice prvkov zoskupené do riadkov. Na rozdiel od maticového editora funguje vždy na báze textových vstupov. Príslušnosť prvkov k interpretáciám unárnych predikátov je vyznačená priamo pri nich. Posledný riadok slúži na pridávanie nových trojíc. K pridaniu však dôjde iba vtedy, ak sú správne vyplnené všetky vstupy v riadku. Tlačidlo s ikonou koša slúži na odstránenie príslušnej trojice.

Pri reprezentáciách funkčných symbolov funguje rozhranie opäť trochu iným spôsobom. Najprv sú automaticky generované všetky n -tice prvkov domény, kde n zodpovedá arite funkčného symbolu. Tieto n -tice sa následne zobrazia v tabuľke a ku každej z nich je sprava priradený jeden ďalší textový vstup, ktorý určuje, akú hodnotu má funkcia nadobúdať pre príslušnú n -ticu. Takýto prístup je zvolený kvôli tomu, že používateľ musí vždy poskytnúť celú definíciu funkcie. Z tohto dôvodu nie je potrebné ani vhodné, aby musel jednotlivé n -tice pridávať manuálne.

Databázový editor je špecifický aj tým, že sa dá ľahko prispôsobiť ľubovoľnej arite, keďže tá priamo súvisí s počtom stĺpcov tabuľky. Pri interpretáciách funkčných symbolov sme sa však rozhodli obmedziť jeho použiteľnosť maximálnou veľkosťou arity 5. Totižto generované tabuľky by už pri vyšších aritách boli príliš veľké a prakticky nepoužiteľné.

Prvok 1	Prvok 2	Prvok 3	
A 	B 	D 	
E	C 	C 	
D 	A 	E	
B 	C 	B 	
nová hodnota 1	nová hodnota 2	nová hodnota 3	

Obr. 3.8: Návrh rozhrania pre databázový editor.

3.5 Editor interpretácií funkcií rozborom prípadov

Zámerom editora interpretácií funkcií rozborom prípadov je umožniť definovať interpretácie funkčných symbolov pomocou dvojíc pozostávajúcich z podmienky kladenej na vstupné argumenty a príslušnej hodnoty, ktorú má funkcia pri splnení tejto podmienky

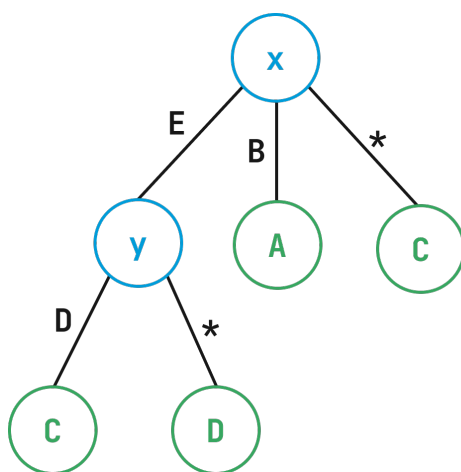
nadobúdať. Príklad takejto definície pre funkciu s dvoma argumentmi (3.1) je uvedený nižšie.

$$f(x, y) = \begin{cases} A, & \text{ak } x = B \\ C, & \text{ak } x = E \text{ a } y = D \\ B & \text{inak.} \end{cases} \quad (3.1)$$

Jednotlivé prípady predstavujú rôzne špecifické podmienky. Napríklad prvá sa vzťahuje iba na argument x , zatiaľ čo v druhej sa zohľadňuje aj y . Aby bola definícia úplná, je potrebné špecifikovať aj prípad, keď nie je splnená žiadna z uvedených podmienok. V príklade je tento prípad označený ako „inak“.

Keďže editor by mal fungovať pre ľubovoľnú aritu funkcie, bolo v prvom rade potrebné zvoliť vhodnú dátovú reprezentáciu takejto štruktúry. Rozhodli sme sa pre špecifický typ stromu, ktorý sme nazvali *strom rozboru prípadov*. Ilustračný príklad možno vidieť na obrázku 3.9. Uzly stromu buď predstavujú argumenty funkcie (označené modrou), alebo konkrétne hodnoty (označené zelenou). Uzol reprezentujúci argument je vnútorný uzol, teda vždy musí mať aspoň jedného potomka, pričom hrany z takéhoto uzla vyjadrujú podmienky na daný argument. Naopak, uzly reprezentujúce hodnoty sú vždy listy stromu.

Každá cesta od koreňa k listu teda určuje konkrétnu kombináciu podmienok vedúcich k výslednej hodnote. Hrany označené hviezdičkou predstavujú všetky ostatné prípady, teda si ich možno predstaviť aj ako zvyšné prvky domény, ktoré neurčujú žiadnu konkrétnu podmienku vychádzajúcu z daného uzla. Podmienkou je, aby z každého uzla vychádzala práve jedna hrana tohto typu.



Obr. 3.9: Strom rozboru prípadov pre Editor interpretácií funkcií rozborom prípadov.

Strom rozboru prípadov nám umožňuje jednoznačne reprezentovať interpretácie funkčných symbolov, a to aj pomocou relatívne malého množstva dát. V porovnaní s ostatnými editormi to predstavuje veľkú výhodu, keďže tie vyžadujú explicitné definovanie celej interpretácie.

Ďalej bolo potrebné pre editor navrhnuť vhodné používateľské rozhranie, ktoré dovolí tvorbu stromu rozboru prípadov. Počas jeho vývoja sme prešli viacerými rôznymi návrhmi. Pre poskytnutie lepšej predstavy o tom, čo viedlo k finálnemu návrhu, sú ďalej jednotlivé verzie v krátkosti popísané.

Prvý návrh Prvý spôsob prezentácie stromu rozboru prípadov možno vidieť na obrázku 3.10. Jednotlivé riadky zodpovedajú cestám v strome od koreňa až po list, pričom podľa vzoru matematického zápisu je hodnota v liste uvedená ako prvá. Všetky editovateľné časti sú reprezentované ako textové vstupy. Pridanie podmienky na ďalší argument funkcie sa realizuje pomocou príslušných tlačidiel v každom riadku. V kontexte stromovej reprezentácie je to rovnaké ako pridať hranu s uzlom reprezentujúcim argument. Vymazať konkrétny riadok je možné pomocou tlačidla s ikonou koša.

Na tejto reprezentácii nám však nevyhovovalo používateľské rozhranie, pretože sa nám nezdalo byť dostatočne intuitívne. Napríklad nebolo zrejmé, ako by mali byť označované jednotlivé argumenty, čo by bolo najmä pre nového používateľa dosť máťúce. Zároveň rozhranie neumožňovalo vykonávať niektoré dôležité úkony, ako napríklad pridávanie nových podmienok.

A if x = B

Add sub-condition ▼

C if x = E

and y = D

D for any other y

C otherwise

Add sub-condition ▼

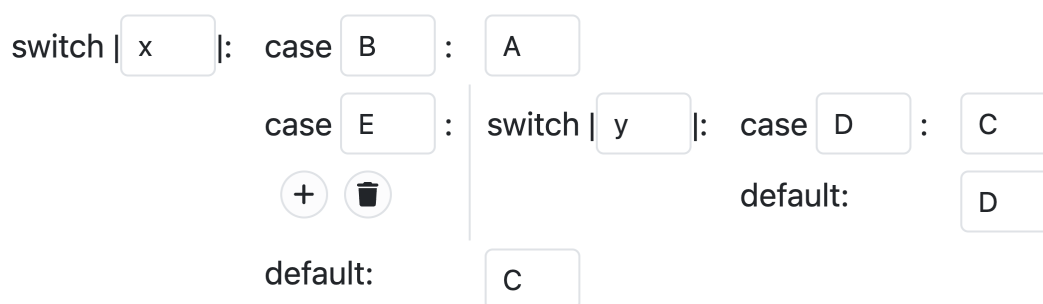
Obr. 3.10: Prvý návrh pre Editor interpretácií funkcií rozborom prípadov.

Druhý návrh Druhý návrh je realizovaný tak, aby veľmi priamočiaro zodpovedal stromu rozboru prípadov. Konkrétny príklad možno vidieť na obrázku 3.11. Uzly reprezentujúce argumenty funkcie sú znázornené kľúčovým slovom „switch“ spolu s textovým vstupom na označenie argumentu. Pre každý takýto uzol sa následne definujú podmienky, ktoré sú znázornené kľúčovým slovom „case“ s príslušným textovým vstupom pre jej hodnotu. Podmienka znázornená kľúčovým slovom „default“ označuje všetky ostatné prvky domény a pridáva sa automaticky, keďže vždy musí mať určenú hodnotu.

Presunutím kurzora na konkrétnu podmienku sa zobrazí menu. V ňom je pomocou tlačidla s ikonou koša možné danú podmienku vymazať, tlačidlo s ikonou + slúži na pridanie novej podmienky. Po jeho stlačení je ešte potrebné určiť, či podmienka bude obsahovať priamo hodnotu, alebo ďalší argument funkcie.

V tejto verzii bola tiež pridaná validácia jednotlivých vstupov. Napríklad sa kontroluje, či používateľ nezadal viac podmienok s rovnakou hodnotou, alebo či nebol ten istý argument použitý už v niektorom rodičovskom uzle.

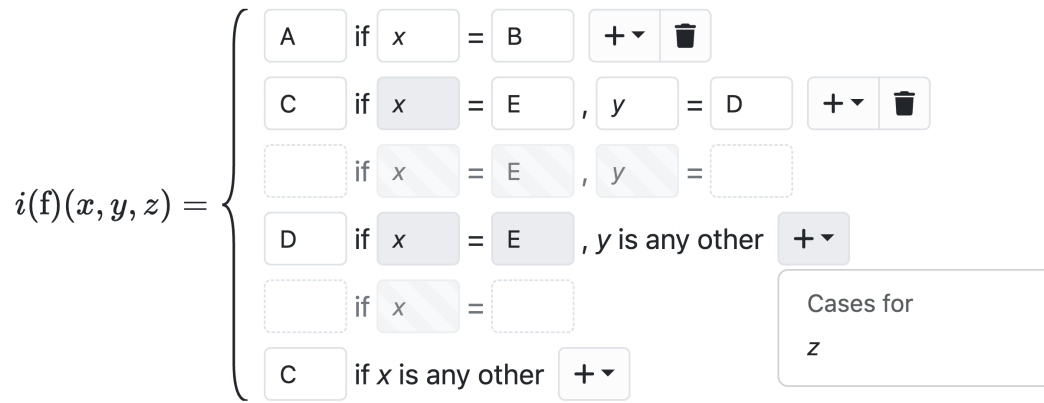
Dôvod, prečo sme nezostali pri tejto reprezentácii, bol najmä ten, že jej vizuálna podoba nezodpovedala matematickej definícii. Napriek tomu nám však poskytla veľmi dobrý základ, z ktorého sme mohli vychádzať pri ďalšom návrhu.



Obr. 3.11: Druhý návrh pre Editor interpretácií funkcií rozborom prípadov.

Tretí návrh Pri finálnom návrhu (obr. 3.12) sme sa snažili, aby čo najvernejšie zodpovedal matematickému zápisu. Spôsob rozdelenia jednotlivých prípadov na riadky je podobný ako pri prvom návrhu. Rozdiel je však v tom, že tlačidlo na bližšiu špecifikáciu podmienky sa nachádza na pravej strane príslušného riadka. Po jeho stlačení sa zobrazí výber argumentov funkcie, ktoré sa v podmienke ešte nevyskytujú. Po výbere sa automaticky pridá nová podmienka s predvyplneným textovým vstupom podľa zvoleného argumentu. Pridanie novej podmienky sa realizuje prostredníctvom menej výrazných, čiastočne priehľadných riadkov. Tie sa v kontexte stromu nachádzajú všade tam, kde je možné pridať ďalšiu podmienku k uzlu reprezentujúcemu argument, pretože ešte neboli vytvorené podmienky na rovnosť so všetkými prvkami domény okrem posledného. Jednotlivé vstupy sú taktiež validované, podobne ako pri druhom návrhu.

Pri takomto spôsobe reprezentácie stromu je však často potrebné zobrazovať jeden uzol reprezentujúci argument naprieč viacerými riadkami. Preto sme sa rozhodli povoliť úpravu tohto argumentu iba v tom riadku, v ktorom sa vyskytuje po prvýkrát. V ostatných riadkoch je príslušný textový vstup deaktivovaný.



Obr. 3.12: Tretí návrh pre Editor interpretácií funkcií rozborom prípadov.

3.6 Dopyty

Komponent dopytov neslúži ako editor, ale reprezentuje samostatnú časť aplikácie určenú na definovanie dopytov, ktorých výsledkom sú všetky ohodnotenia individuových premenných spĺňajúce otvorenú formulu logiky prvého rádu zadanú používateľom. Obrázok 3.13 zobrazuje návrh rozhrania a jeho umiestnenie v aplikácii.

V kontexte aplikácie má komponent štandardné rozhranie. Na vrchu sa nachádza jeho názov, pod ním sa definujú dopyty. Tlačidlom „Pridať dopyt“ v dolnej časti je možné pridávať nové, prázdne dopyty. Návrh rozhrania konkrétneho dopytu je na obrázku zobrazený pod nadpisom „Dopyty“.

Na definovanie dopytu je najprv potrebné deklarovať voľné premenné, podľa ktorých sa budú generovať ohodnotenia. Tie sa zadávajú do prvého textového vstupu zľava a musia byť oddelené čiarkou. Druhý textový vstup hneď vedľa slúži na definovanie otvorenej formuly, ktorú chceme vyhodnotiť. Oba vstupy sú validované. Pri premenných sa kontroluje správny formát, ako aj to, či množina deklarovaných premenných je práve množinou voľných premenných danej formuly. Ak nie, používateľovi sa pod vstupom zobrazí chybová správa s informáciou o konkrétnych premenných, ktoré podmienku nespĺňajú. Pri formulách sa kontroluje ich správny zápis, teda napríklad to, či všetky použité mimologické symboly patria do definovaného jazyka a či sa používajú so správnym počtom argumentov.

Ak sú oba vstupy vyplnené správne, používateľ môže stlačiť tlačidlo „Vyhodnotiť“ úplne napravo. Tým sa dopyt vyhodnotí, teda sa vygenerujú všetky ohodnotenia, ktoré spĺňajú danú otvorenú formulu. Následne sa zobrazí počet výsledkov spolu s ich zoznamom. Každý riadok predstavuje jedno ohodnotenie a stĺpce zodpovedajú jednotlivým voľným premenným. Zoznam je navyše stránkovaný, takže aj pri väčšom množstve ohodnotení sa zobrazí iba obmedzený počet výsledkov. Prechádzanie strán zoznamu je umožnené tlačidlami na jeho spodku.

Problém, ktorý môže nastať je, ak používateľ po vyhodnotení dopytu nejakým spô-

Jazyk

Pravdivosť formúl

Štruktúra

Voľné premenné

Otvorená formula

Vyhodnotiť

Výsledky dopytu

Spolu 5 výsledkov

premenná_1	premenná_2
A	B
B	C
D	D

Predošlá strana
Ďalšia strana

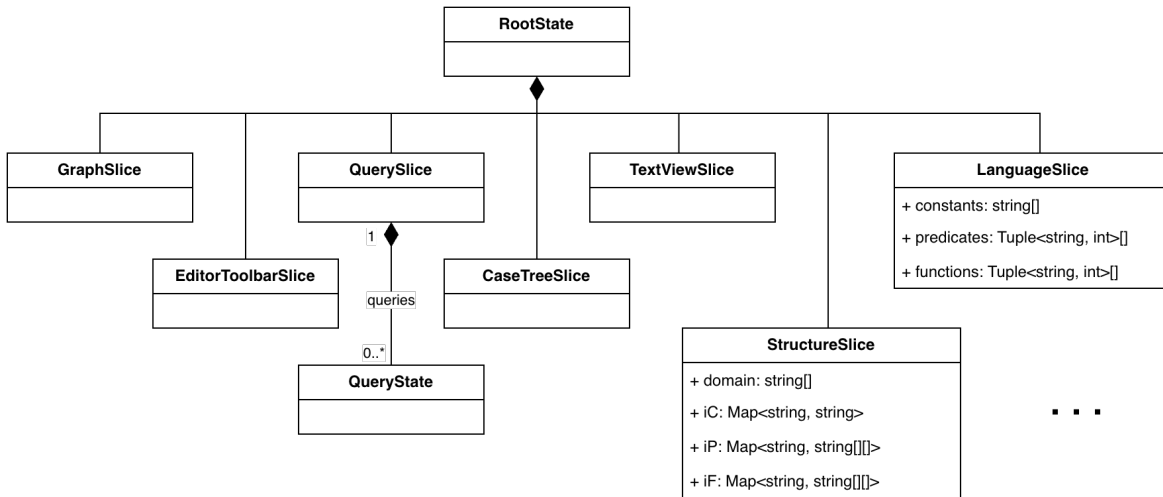
Pridať dopyt

Obr. 3.13: Návrh rozhrania a umiestnenia pre komponent dopytov.

sobom upraví štruktúru. To totiž často znamená, že vygenerované ohodnotenia už nie sú aktuálne.

Jedným z riešení bolo pri každej zmene štruktúry všetky existujúce výsledky skryť. Na ich opätovné zobrazenie by bolo potrebné opäť stlačiť tlačidlo na vyhodnotenie. Tento variant nám však úplne nevyhovoval, pretože sme chceli, aby boli výsledky stále k dispozícii. Pre používateľa môžu byť totiž nápomocné, najmä počas úprav samotnej štruktúry. Zároveň sme nechceli vyhodnotenie dopytu spúšťať automaticky po každej zmene, keďže pri väčšom množstve súčasne otvorených dopytov s veľkým počtom ohodnotení by takýto prístup mohol výrazne zhoršovať výkon aplikácie. Navyše by si používateľ ani nemusel uvedomiť, že sa výsledok dopytu zmenil.

Nakoniec sme sa rozhodli zvoliť prístup, pri ktorom sa výsledky po upravení štruktúry neskryjú, ale sú označené ako potenciálne neaktuálne pomocou varovnej správy zobrazenej v hlavičke výsledkovej tabuľky. Na odstránenie tohto varovania ich používateľ musí opäť vyhodnotiť.



Obr. 3.14: Triedny diagram nových súčastí stavu.

3.7 Nový stav aplikácie

V pôvodnej implementácii je stav organizovaný do viacerých logických celkov, ktoré zodpovedajú jednotlivým častiam aplikácie. Medzi ne patria napríklad `LanguageSlice` a `StructureSlice`, predstavujúce jazyk, resp. štruktúru. Všetky časti stavu sú spojené do centrálneho stavu aplikácie (`RootState`).

Stavy našich rozšírení sú tiež integrované do centrálneho stavu aplikácie spolu s jeho existujúcimi časťami. Triedny diagram vybraných nových súčastí stavu spolu s novými typmi atribútov `StructureSlice` a `LanguageSlice` možno vidieť na obrázku 3.14. V nasledujúcich sekciách sa venujeme podrobnejšiemu návrhu stavu jednotlivých rozšírení. Tam, kde je to vhodné, doplníme návrhy aj diagramom znázorňujúcim ich dátový model.

3.7.1 Spoločné rozhranie editorov

Časť stavu spoločného rozhrania editorov (`EditorToolbarSlice`) je potrebná na ukladanie informácií o aktuálne otvorenom editore a jednotlivých nastaveniach filtrovania pre každý predikát a funkciu. Konkrétne sa jedná o stav zaškrtnutia tlačidla domény, názvy zaškrtnutých unárnych predikátov ako aj zoznam prvkov domény označených v komponente filtra domény. Záznam o tomto stave je jednoznačne identifikovaný kombináciou príslušného symbolu a jeho typu.

3.7.2 Textový pohľad

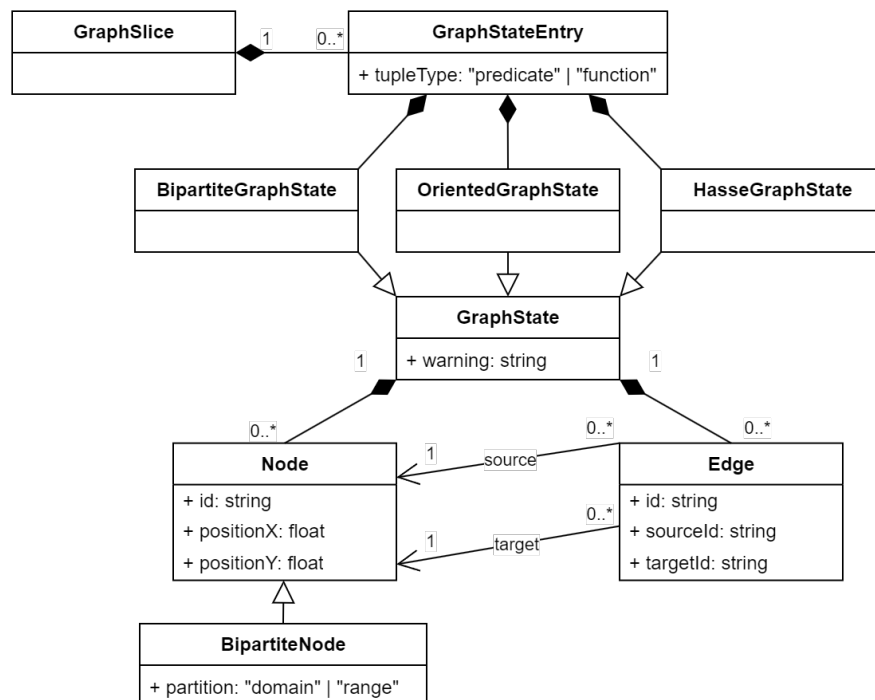
V pôvodnej implementácii časti stavu pre jazyk a štruktúru uchováva väčšinu svojich dát výhradne v textovej podobe. Teda napríklad pri štruktúre je interpretácia

konkrétneho predikátu reprezentovaná ako text, ktorý používateľ zadal do príslušného textového vstupu.

Tento spôsob reprezentácie by bol však pri implementácii našich rozšírení veľmi nepraktický a neefektívny, keďže by si vyžadoval neustále premieňanie medzi textovou a štruktúrovanou formou interpretácie používanou v jednotlivých editoroch. Na druhej strane, pre správne fungovanie textových vstupov je potrebné, aby bol aj naďalej ukladaný ich presný obsah, keďže používateľ môže zadať aj syntakticky nesprávne vstupy, ktoré sa nedajú priamo previesť do štruktúrovanej formy.

Rozhodli sme sa preto zaviesť novú časť stavu (**TextViewSlice**) určenú na ukladanie obsahu všetkých textových vstupov spomínaných častí aplikácie. Každý takýto vstup je reprezentovaný pomocou záznamu, ktorý obsahuje okrem samotného textového reťazca aj informáciu o type reprezentovaných dát (napr. či ide o individuové konštanty alebo interpretáciu funkčného symbolu). Jednotlivé záznamy sú unikátne identifikované typom reprezentovaných dát a v prípade interpretácií konštánt, predikátov a funkcií aj ich názvom.

V pôvodných častiach stavu sú všetky textové reprezentácie nahradené vhodnou štruktúrovanou formou. Napríklad v prípade interpretácií predikátov ide o zoznam jednotlivých n -tíc prvkov domény, kde n je arita predikátu.



Obr. 3.15: Triedny diagram pre grafové editory.

3.7.3 Grafové editory

V časti stavu pre grafové editory sa ukladajú dáta jednotlivých grafových reprezentácií. Návrh dátového modelu je znázornený na obrázku 3.15. Každému binárnemu predikátu a unárnej funkcii zodpovedá jeden záznam (**GraphStateEntry**) v slovníkovom type **GraphSlice**, ktorý je jednoznačne identifikovaný kombináciou názvu a typu príslušného symbolu.

Každý objekt typu **GraphStateEntry** obsahuje typ príslušného symbolu (atribút **tupleType**) a stav (**GraphState**) pre jednotlivé typy grafov. Tento stav zahŕňa zoznam vrcholov (**Node**) a hrán (**Edge**). Hrana je navyše asociovaná s dvojicou vrcholov, pričom jeden je počiatočný (**source**) a druhý koncový (**target**). Vrcholy obsahujú svoju pozíciu v zobrazení grafu (atribúty **positionX** a **positionY**) a unikátny identifikátor (atribút **id**) používaný na jednoznačné priradenie počiatočného a koncového vrcholu hrany s konkrétnou inštanciou objektu typu **Node**. Vrchol bipartitného grafu (**BipartiteNode**) rozširuje **Node** o atribút **partition**, ktorý určuje jeho príslušnosť k jednej z dvoch skupín.

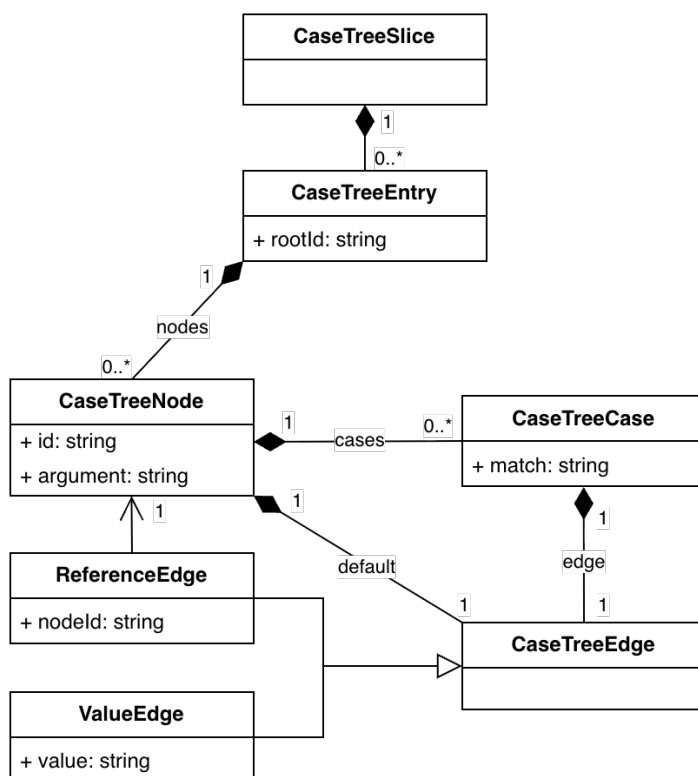
3.7.4 Maticový a databázový editor

Rovnako ako pri grafových editoroch, aj v maticovom a databázovom editore je záznam o ich stave jednoznačne identifikovaný pomocou názvu a typu predikátového alebo funkčného symbolu. V prípade databázového editora je stav ukladaný ako zoznam hodnôt zadanych do textových vstupov pre jednotlivé n -tice. Naopak maticový editor pre každú n -ticu zaznamenáva buď stav zaškrtávacieho políčka, alebo hodnotu textového vstupu, v závislosti od typu reprezentácie.

3.7.5 Editor interpretácií funkcií rozborom prípadov

Stav editora interpretácií funkcií rozborom prípadov je podobný stromovej štruktúre opísanej v podkapitole 3.5. Rozdielom však je, že používame iba uzol predstavujúci argument funkcie. Informáciu, ktorú niesol uzol predstavujúci hodnotu sme presunuli do hrán stromu. Návrh dátového modelu editora možno vidieť na obrázku 3.16. Každému funkčnému symbolu zodpovedá jeden záznam (**CaseTreeEntry**) v slovníkovom type **CaseTreeSlice**, ktorý je jednoznačne identifikovaný príslušným symbolom.

V objekte typu **CaseTreeEntry** sú uložené jednotlivé uzly (**CaseTreeNode**) spolu s identifikátorom koreňového uzla (atribút **rootId**). Každý uzol je tvorený argumentom (atribút **argument**), ktorý reprezentuje, unikátnym identifikátorom (atribút **id**), zoznamom z neho vychádzajúcich podmienkových hrán (**CaseTreeCase**) a hranou označujúcou všetky ostatné prípady (**CaseTreeEdge**). Abstraktný objekt typu **CaseTreeEdge** špecializujú dva typy hrán. Jednou z nich je referenčná hrana (**ReferenceEdge**), ktorá



Obr. 3.16: Triedny diagram pre editor interpretácií funkcií rozborom prípadov.

si uchováva identifikátor druhého uzla, teda ďalšej inštancie **CaseTreeNode**. Druhou je hodnotová hrana (**ValueEdge**), tá priamo ukladá samotnú hodnotu.

Podmienkové hrany (**CaseTreeCase**) predstavujú jednotlivé podmienky kladené na argument príslušného uzla. Preto obsahujú hodnotu (atribút `match`), s ktorou sa má príslušný argument porovnať a aj samotnú hranu (**CaseTreeEdge**).

3.7.6 Dopyty

Časť stavu pre dopyty (**QuerySlice**) obsahuje zoznam jednotlivých dopytov. Každý dopyt sa skladá z textovej reprezentácie zoznamu voľných premenných a otvorenej formuly.

Kapitola 4

Implementácia

V tejto kapitole sa zameriame na vybrané časti implementácie. Tie majú priblížiť hlavné problémy riešené počas vývoja a zároveň vysvetliť zvolené prístupy a ich realizáciu pomocou použitých technológií. Cieľom je taktiež poskytnúť základný prehľad implementácie, ktorý uľahčí orientáciu v systéme pri jeho prípadnom budúcom rozšírení.

4.1 Výber technológií

Nová verzia prieskumníka štruktúr je plne implementovaná v jazyku TypeScript. Používateľské rozhranie je vytvorené pomocou knižnice React a na správu globálneho stavu sa používa knižnica Redux (všetky sú opísané v podkapitole 1.3). Prirodzene sme sa teda rozhodli pokračovať v používaní týchto technológií aj pri tvorbe našich rozšírení. TypeScript sa ukázal ako veľmi nápomocný aj počas oboznamovania sa s existujúcou implementáciou, keďže vďaka definovaným typom bolo výrazne jednoduchšie porozumieť očakávaným tvarom objektov alebo argumentov funkcií.

Ďalej bolo potrebné vybrať nástroj na implementáciu grafových editorov. Jednou z možností bola knižnica React Diagrams [1], ktorú na implementáciu grafového zobrazenia použil vo svojej bakalárskej práci Miroslav Baluch [7]. Napriek pomerne širokým možnostiam tejto knižnice sme sa ju však rozhodli nepoužiť, keďže už nie je aktívne vyvíjaná ani udržiavaná.

Vzhľadom na spomenuté technológie sme mali ďalej na výber medzi knižnicami React Flow [2] a Reagraph [3]. Obe sú v súčasnosti populárne a pravidelne aktualizované, avšak knižnica Reagraph na rozdiel od React Flow ponúka len veľmi obmedzené možnosti prispôsobenia vrcholov a hrán, ktoré by nepostačovali na implementáciu požadovanej funkcionality. Ako najvhodnejšie riešenie sme preto napokon zvolili knižnicu React Flow, keďže podporovala už spomenuté technológie a zároveň poskytovalo dostatočné množstvo konfigurácie na implementáciu všetkých navrhnutých funkcionalít. Veľkou výhodou bola aj kvalitná dokumentácia doplnená sadou interaktívnych príkla-

dov, ktoré výrazne uľahčovali oboznámenie sa s dostupnými možnosťami.

4.2 Integrácia nových editorov interpretácií

V tejto podkapitole priblížime spôsob, akým sme nové editory interpretácií navrhnuté v podkapitolách 3.3 až 3.5 zakomponovali do už existujúcej aplikácie.

4.2.1 Organizácia

Súborovú štruktúru pôvodnej aplikácie sme v zásade nemenili. Držali sme sa stanovenej konvencie, podľa ktorej je každá funkcionálna organizovaná vo vlastnom podpriechniku v `src/features`. Zdrojové súbory pre jednotlivé nástroje preto ukladáme do samostatných podpriechnikov pomenovaných podľa daného nástroja (napr. `graphView` pre grafové editory). Obsah týchto priechnikov je rôzny, no každý obsahuje aspoň hlavný React komponent pre daný nástroj (napr. `MatrixView` pre maticový editor) a súbor definujúci jeho Redux stav (napr. `databaseViewSlice.ts` pre databázový editor). V rámci stavu sú spravidla implementované reducers a potrebné selektory.

4.2.2 Integrácia komponentov editorov

Predošlá verzia aplikácie používala na zobrazovanie interpretácií textové vstupy, tie boli obalené v komponente `InterpretationInput` a následne používané naprieč aplikáciou. Pre náš účel však potrebujeme riešenie, ktoré bude vedieť vybrať a zobrazíť vhodný komponent na základe aktuálne zvoleného editora.

Tento problém rieši nový komponent `InterpretationEditor`. Ten zaobahuje všetky editory vrátane pôvodného. Jedným z jeho vstupných parametrov je názov reprezentovaného symbolu. Na základe tohto názvu získa zo stavu typ editora, ktorý sa má zobraziť. Ak ide o textový editor, zobrazuje sa pôvodný komponent `InterpretationInput`. V opačnom prípade sa vykreslí nový komponent `DrawerEditor`.

`DrawerEditor` integruje spoločné rozhranie editorov (navrhnuté v podkapitole 3.2) pozostávajúce z ďalších troch častí. Jednou z nich je komponent panela nástrojov `EditorToolbar`. Ten implementuje rozhranie filtrovania. Ďalším je `EditorError`, predstavujúci chybový panel. Tretou časťou je samotný komponent editora, ktorý je určený na základe získaného typu.

4.2.3 Synchronizácia stavu

Jednotlivé editory reprezentujú interpretácie symbolov vo svojom stave rôznym spôsobom. To však predstavuje problém, keďže zmeny stavu jednotlivých editorov je po-

```
1 extraReducers(builder) {  
2   builder.addCase(updateInterpretationPredicates, (state,  
    action) => {  
3     if (action.meta.source === "databaseView") return;  
4  
5     const { key, value } = action.payload;  
6     syncInterpretation(key, "predicate", value, state);  
7   });  
8 }
```

Ukážka programu 4.1: Extra reducer používaný pri aktualizácii databázového editora.

trebné synchronizovať, a to ideálne automaticky. Teda každá zmena vykonaná v jednom editore by sa mala okamžite premietnuť aj do stavu ostatných editorov.

Riešením je zaviesť jednotné miesto a formát ukladania interpretácií, ktorý budú všetky editory zdieľať a vedieť spracovať. Na tento účel sme sa rozhodli použiť interpretácie uložené v rámci stavu štruktúry. V nasledujúcej časti popíšeme spôsob, akým je implementované zdieľanie týchto dát medzi jednotlivými editormi.

Aktualizácia interpretácie konkrétneho symbolu uloženého v stave štruktúry sa realizuje pomocou príslušných akcií:

- `updateInterpretationPredicates`: akcia na aktualizáciu predikátu
- `updateInterpretationFunctions`: akcia na aktualizáciu funkcie

Tieto akcie okrem svojho typu nesú aj názov daného symbolu (atribút `key`) a samotnú aktualizovanú interpretáciu (atribút `value`).

Každý editor má v súvislosti s týmito akciami dve úlohy. Prvou je vyvolať príslušnú akciu v momente, keď sa v jeho pohľade nejakým spôsobom zmení samotná interpretácia. To vyžaduje okrem iného aj konverziu medzi jeho internou reprezentáciou a formátom používaným v štruktúre.

Druhou je reagovať na tieto akcie vo svojom vlastnom reduceri a na základe informácií, ktoré nesú, aktualizovať svoj stav. Na tento účel bolo potrebné využiť mechanizmus knižnice Redux, ktorý umožňuje jednému reduceru reagovať na akcie iného. V tomto prípade teda reducery editorov reagujú na spomínané akcie štruktúry.

Ukážka takéhoto reducera (4.1) demonštruje spracovanie zmeny interpretácie predikátu v databázovom editore. Reducer je definovaný v rámci funkcie `extraReducers`, ktorá dostáva ako argument objekt `builder`. Pomocou metódy `addCase` sa určí akcia (prvý argument), pri ktorej sa má príslušný reducer zavolať (druhý argument). V tele reducera sa najprv prostredníctvom atribútu `meta.source` kontroluje pôvod akcie. Ten sa vždy nastavuje pri jej vyvolaní. Je to dôležité najmä z toho dôvodu, aby

databázový editor zbytočne nespracovával akcie, ktoré sám vyvolal. Následne sa volá funkcia `syncInterpretation`, implementujúca logiku prevodu interpretácie do podoby používanej daným editorom.

4.3 Implementácia grafových editorov

Táto podkapitola približuje jednotlivé časti implementácie grafových editorov. Postupne prejdeme štruktúrou stavu, spôsobom jeho využitia a nakoniec sa pozrieme na integráciu komponentov knižnice React Flow.

4.3.1 Štruktúra stavu

Stav grafových editorov sme štruktúrovali podľa návrhu opísaného v časti 3.7.3. Stav vrcholov (**Node**) a hrán (**Edge**) sme sa však museli prispôbiť knižnici React Flow. Tá totiž očakáva špecifický typ objektu popisujúci stav vrcholov a hrán, ktorý nie je odporúčané priamo rozširovať o vlastné atribúty. Namiesto toho je na tento účel vyčlenený atribút `data`, ktorého typ možno ľubovoľne špecifikovať. V prípade vrcholov ho využijeme na zaznamenanie momentálneho variantu ich zobrazenia, pomocou voliteľných boolovských atribútov `error`, `ghost`, `hatched` a `leftover` (bližšie opísané v časti 3.3.1 venovanej orientovanému grafu). Konkrétne atribúty `error` a `leftover` zodpovedajú návrhu chybových vrcholov 3.3(a), resp. 3.3(b), a atribúty `ghost` a `hatched` zodpovedajú návrhom 3.2(c), resp. 3.2(d). Vrchol bipartitného grafu obsahuje navyše atribút `origin`, vyjadrujúci jeho príslušnosť do skupiny (`'domain'` alebo `'range'`).

4.3.2 Práca so stavom

Jednotlivé dáta zo stavu sú komponentom grafov sprístupnené prostredníctvom selektorov. To dovoľuje oddeliť transformáciu dát od implementácie používateľského rozhrania. Jednotlivé selektory je navyše možné navzájom kombinovať, čo uľahčuje ich organizáciu a znižuje duplicitu logiky. Všetky selektory súvisiace so stavom grafov sa nachádzajú v súbore `graphSlice.ts`.

Napríklad úlohou selektora `selectNodes` je pre konkrétnu reprezentáciu získať zoznam vrcholov, aplikovať na ne aktuálne doménové filtre a výsledok vrátiť v podobe zoznamu. Ako vstupné argumenty prijíma typ symbolu interpretácie (`'predicate'` alebo `'function'`), jeho názov a aj typ samotnej reprezentácie (napr. `'hasse'`). Na základe týchto informácií dokáže zo stavu vybrať príslušné vrcholy. Na získanie domény po aplikovaní všetkých filtrov sa používa ďalší selektor `selectRelevantDomain`. Pomocou neho sa zo zoznamu vrcholov ešte odstránia tie, ktoré nezodpovedajú žiadnemu prvku vo vyfiltrovannej doméne.

Podobne funguje aj selektor `selectEdges`, ktorého úlohou je vrátiť zoznam hrán priamo použiteľný pri zobrazení grafu. Napríklad pri Hasseho diagrame je v tomto selektore potrebné odstrániť nadbytočné hrany (podľa pravidiel opísaných v sekcii 1.2.2), keďže v jeho stave sú explicitne uložené všetky hrany reprezentujúce interpretáciu.

Aktualizácia stavu prebieha prostredníctvom reducerov a k nim prislúchajúcich akcií. Zmeny týkajúce sa stavu vrcholov a hrán sú reprezentované akciami `nodesChanged` a `edgesChanged`. Zmena môže zahŕňať napríklad posunutie vrcholu alebo označenie hrany. Pri vytvorení novej hrany knižnica `React Flow` vyvolá akciu `nodesConnected`, ktorej súčasťou sú aj identifikátory (odvodené z príslušných prvkov domény) oboch vrcholov. Akcia `leftoverDeleted` slúži na odstránenie chybného vrcholu. Vyvoláva sa pri kliknutí na tlačidlo koša príslušného vrcholu, popísaného v sekcii 3.3.1 venovanej návrhu vrcholov.

Dôležitou je aj akcia `syncGraphView`, používaná pri synchronizácii grafových editorov so samotnou štruktúrou. Využíva sa napríklad pri pridaní nového binárneho predikátu alebo unárnej funkcie, keďže vtedy je potrebné inicializovať stav príslušných grafových editorov.

4.3.3 Komponenty

Hlavný komponent, ktorý obaľuje komponenty konkrétnych reprezentácií je `GraphView`. Jeho úlohou je na základe poskytnutého typu grafu vykresliť komponent aktuálne zvolenej reprezentácie. Zobrazenie každej reprezentácie je implementované v rámci jej vlastného komponentu, konkrétne ide o `OrientedGraph`, `BipartiteGraph` a `HasseDiagram`. Tieto komponenty následne obaľujú komponent `ReactFlow` poskytovaný knižnicou, ktorý sa využíva na vykreslenie samotného rozhrania grafu.

Úlohou jednotlivých komponentov je získavať potrebné dáta na vykreslenie grafu zo stavu príslušnej reprezentácie, ako sú napríklad vrcholy a hrany, prostredníctvom vyššie spomínaných selektorov. Zároveň sa v nich definuje konfigurácia komponentu `ReactFlow`, špecifická pre daný typ grafu. Príkladom možnosti, ktorú knižnica pri konfigurácii ponúka, je funkcia `isValidConnection`. Po jej špecifikovaní ju knižnica volá pri každom pokuse o vytvorenie novej hrany, čo poskytuje možnosť implementovania vlastnej validačnej logiky pre vytváranie hrán. Táto funkcionálnosť sa využíva napríklad v komponente `HasseDiagram`, kde sa kontroluje, či by pridaním hrany ostali zachované všetky podmienky čiastočného usporiadania.

V rámci konfigurácie je tiež potrebné špecifikovať komponenty, ktoré bude `React Flow` používať na vykresľovanie vrcholov a hrán. V našom prípade sme na to zadefinovali komponenty `PredicateNode` a `DirectEdge`, ktoré definujú konkrétne zobrazenie a používateľské rozhranie vrcholov a hrán grafu. Pomocou komponentu `PredicateNode` je implementovaná napríklad funkcionálnosť vytvárania hrán alebo zobrazovania panela

s farbami unárnych predikátov, ktoré sme navrhli v časti 3.3.1.

4.4 Refaktorizácia stavu pôvodnej aplikácie

Ako bolo uvedené v podkapitole 3.7 venovanej návrhu nového stavu aplikácie, časti stavu `StructureSlice` a `LanguageSlice`, predstavujúce štruktúru a jazyk, uchovávali výhradne obsah svojich textových vstupov. Každý z týchto vstupov však predstavuje odlišný typ dát, ktorý si vyžaduje špecifický formát zápisu, ako aj vlastnú validáciu a prevod do štruktúrovanej podoby. Z toho dôvodu bol každý typ textového vstupu asociovaný s príslušnou akciou na jeho aktualizáciu a selektorom. V tomto selektore prebiehala ich premena na štruktúrovanú podobu spolu so syntaktickou a sémantickou validáciou.

Prvým krokom pri refaktorizácii bolo v pôvodných častiach stavu nahradiť textové reprezentácie ich štruktúrovanou formou a následne týmto zmenám prispôbiť aj príslušné selektory. To si vyžadovalo zo selektorov odstrániť časť zodpovedajúcu za premenu z textovej na štruktúrovanú podobu spolu so syntaktickou validáciou. Sémantická kontrola v selektoroch ostala zachovaná. Zároveň bolo potrebné upraviť akcie a k nim prislúchajúce reducery tak, aby namiesto textových vstupov pracovali priamo so štruktúrovanými dátami.

Ďalším krokom bolo vytvorenie novej časti stavu `TextViewSlice` určenej na ukladanie obsahu jednotlivých textových vstupov. Na aktualizáciu stavu konkrétneho textového vstupu sa používa funkcia `updateTextView`. Jej úlohou je v prvom rade na základe typu textového vstupu vykonať syntaktickú kontrolu vstupného reťazca a následne vyvolať akciu `textViewChanged`, ktorá zabezpečuje aktualizáciu obsahu textového vstupu, ako aj prípadnej chybovej správy s ním spojennej. Ak je reťazec syntakticky v poriadku, nasleduje jeho transformácia do štruktúrovanej podoby a vyvolanie jednej z refaktorizovaných akcií zodpovedných za aktualizáciu príslušného typu dát.

Keďže textové vstupy už nie sú jediným spôsobom, ako meniť interpretáciu predikátového alebo funkčného symbolu, je potrebné ich obsah v tomto prípade aktualizovať aj pri každej inde vyvolanej zmene. To je riešené rovnakým spôsobom, ako bolo už opísané v časti 4.2.2 venovanej integrácii komponentov editorov. Teda pomocou extra reducera, ktorý reaguje na vyvolanie akcie predstavujúcej aktualizáciu interpretácie. Tieto akcie však nesú údaje v štruktúrovanej podobe a je ich pred použitím potrebné vhodne previesť do textovej formy.

Na konfiguráciu toho, ako správne spracovať jednotlivé typy dát, sme pre každý zadefinovali popisný objekt. Príklad takéhoto objektu pre interpretáciu predikátov je uvedený v ukážke 4.2. Jeho jednotlivé atribúty sú využívané v už opísaných častiach implementácie textových vstupov.

```

1 predicate_interpretation: {
2     parse: (value) => parseTuples(value),
3     toText: (struct) =>
4         struct.map((tuple) => `(${tuple.join(",")})`).join(", "),
5     validate: selectValidatedPredicate,
6     syncActionCreator: updateInterpretationPredicates,
7     payloadType: "key",
8 }

```

Ukážka programu 4.2: Konfiguračný objekt popisujúci interpretáciu predikátov.

Konkrétne atribútom `parse` špecifikujeme funkciu používanú pri konverzii z textovej na štruktúrovanú formu. Naopak atribút `toText` určuje funkciu na konverziu štruktúrovanej formy do textovej. Atribút `validate` reprezentuje selektor, pomocou ktorého je možné zistiť, či je štruktúrovaná reprezentácia sémanticky validná, čo sa používa v komponentoch textových vstupov na získanie kompletnej chybovej správy. Ďalej `syncActionCreator` predstavuje akciu, na ktorej vyvolanie je potrebné reagovať pri synchronizácii textového vstupu so štruktúrovanou časťou stavu. Posledný atribút `payloadType` určuje, či je nutné pri použití selektora `validate` alebo akcie `syncActionCreator` špecifikovať aj identifikátor (kľúč). V prípade interpretácií symbolov individuových konštánt, predikátov a funkcií je týmto kľúčom samotný symbol. Naopak pri typoch dát vyskytujúcich sa v stave iba raz (napr. doména) nie je nutné uvádzať žiadny identifikátor.

4.5 Integrácia s Logickým pracovným zošitom

Naša aplikácia je do Logického pracovného zošita integrovaná ako knižnica, ktorá poskytuje rozhranie pozostávajúce z funkcie `prepare`, používanej pri inicializácii vlozenej aplikácie, a komponentu `AppComponent`, predstavujúceho koreňový komponent používateľského rozhrania. Požiadavky na ich fungovanie boli implementované už v rámci bakalárskej práce Jozefa Filipa [13]. Pre zakomponovanie našich rozšírení bolo potrebné doplniť už existujúcu funkcionálnosť ukladania a načítania stavu vlozenej aplikácie, realizovanú pomocou dvoch funkcií definovaných v súbore `importExportUtils.ts`.

Na exportovanie momentálneho stavu sa používa funkcia `getAppStateToExport`. Jej úlohou je vrátiť objekt, ktorý Logický pracovný zošit následne prevedie do formátu JSON a uloží. Keďže samotný Redux stav predstavuje tiež iba obyčajný objekt, v pôvodnej implementácii bol jednoducho exportovaný celý stav aplikácie. To však v našom prípade nie je potrebné. Konkrétne stav maticového editora, databázového editora a textového pohľadu sme sa rozhodli vôbec neukladať, keďže ho možno odvodiť z

interpretácií uložených v stave štruktúry. Z rovnakého dôvodu sa aj pri grafových editoroch ukladá iba pozícia jednotlivých vrcholov, pričom informácie o hranách sa opäť odvodzujú zo stavu štruktúry. Týmito optimalizáciami sa nám podarilo výrazne znížiť objem ukladaných dát. Ostatné spomínané rozšírenia ukladajú celý svoj stav.

Pri otvorení aplikácie môže byť dostupný počiatočný stav (ak bol predtým uložený), jeho načítanie je zabezpečené funkciou `importAppState`. Proces načítania sme sa rozhodli rozšíriť o krok validácie vstupných dát, ktorý je realizovaný knižnicou `Zod` [6]. Tá umožňuje definovať schémy opisujúce očakávanú štruktúru objektov. Každú exportovanú časť stavu bolo potrebné pomocou takejto schémy opísať a spojiť do jednej centrálnej schémy `serializedAppStateSchema`. Na overenie typu konkrétneho objektu na základe tejto schémy poskytuje knižnica metódu `parse`. Ak vstupné dáta nespĺňajú požadovaný typ, metóda vráti chybovú hlášku. V takomto prípade sa používateľovi zobrazí varovanie v rozhraní aplikácie, informujúce o tom, že niektoré časti stavu sa nepodarilo načítať. V prípade, že validácia prebehla úspešne, sú jednotlivé časti priamo načítané alebo odvodené.

4.6 Vylepšenia Henkinovej-Hintikkovej hry

Reimplementovaná verzia Henkinovej-Hintikkovej hry spočiatku neobsahovala všetku požadovanú funkcionality. Kľúčovou zmenou jej fungovania bolo zavedenie znáhodnenia v situáciách, keď si hra vyberá podformulu alebo ohodnotenie otvorenej formuly. Prvé pokusy o implementáciu tejto funkcionality však odhalili viaceré nedostatky.

Niektoré z nich spôsobovali výrazné spomalenie a zhoršenie responzivnosti celej aplikácie pri formulách s väčším počtom vnorených kvantifikátorov, čo negatívne ovplyvňovalo používateľský zážitok. Iné zas v určitých okrajových prípadoch viedli až k pádu samotnej aplikácie. Preto bolo najskôr potrebné výrazne refaktORIZOVAŤ existujúcu implementáciu, čo zároveň prispelo k lepšiemu pochopeniu jej fungovania.

Keďže hra implementuje väčšinu svojej logiky v selektoroch, zmeny zahŕňali najmä ich sprehľadnenie, odstránenie zbytočných častí, ako aj úpravu komponentov, ktoré ich využívajú. Ukázalo sa tiež, že selektory zodpovedné za správne resetovanie hry pri zmene štruktúry nepočítali so všetkými možnými situáciami, v ktorých sa hra môže nachádzať, a bolo potrebné tieto prípady vhodne ošetriť. Celkovo moje úpravy priniesli pri zložitejších formulách až desaťnásobné zlepšenie výkonu (z desiatok na jednotky milisekúnd), čo je pre používateľa značne citelné.

Po spomínaných zmenách sa nám pomerne jednoducho podarilo implementovať aj samotné znáhodnenie. Následne sme sa hru rozhodli doplniť o ďalšie vylepšenia. Jedným z nich bolo zjednotenie spôsobu výpisu matematických výrazov v dialógu hry. Pôvodná verzia kombinovala používanie Unicode znakov s prehľadnejším zápi-

som pomocou KaTeX-u, pričom tieto zápisy boli generované naprieč viacerými zdrojovými súbormi. Všetka logika generovania zápisov bola preto presunutá do súboru `messageBubbleFactories.ts`, ktorý obsahuje funkcie na generovanie rôznych typov výrazov výhradne vo formáte KaTeX, využívaných jednotlivými komponentmi hry. Okrem toho bolo pridaných viacero menších vylepšení používateľského rozhrania, ako aj samotného dialógu hry. Porovnanie rozhrania novej verzie s predchádzajúcou na krátkej časti hry možno vidieť na obrázku 4.1.

[Change](#) $\mathcal{M} \not\models (figúrka(x) \vee políčko(x)) [e(x/\phi)]$

You assume that $\mathcal{M} \not\models (figúrka(x) \vee políčko(x)) [e(x/\phi)]$

Then $\mathcal{M} \not\models figúrka(x) [e(x/\phi)]$

[Continue](#)

You assume that $\mathcal{M} \not\models figúrka(x) [e(x/\phi)]$

You lose, $\mathcal{M} \models figúrka(x) [e(x/\phi)]$, since $(\phi) \in i(figúrka)$

You could have won, though. Your initial assumption that $\mathcal{M} \models \forall x((figúrka(x) \vee políčko(x)) \rightarrow (farba(x) = biela \vee farba(x) = čierna)) [e]$ was correct. Find incorrect intermediate answers and correct them! You can use **change button** next to your answers for that.

(a) Pôvodná verzia

[Change](#) $\mathcal{M} \not\models (figúrka(x) \vee políčko(x)) [e(x/a1)]$

Let's assume that $\mathcal{M} \not\models (figúrka(x) \vee políčko(x)) [e(x/a1)]$

Then simultaneously:

- $\mathcal{M} \not\models figúrka(x) [e(x/a1)]$
- $\mathcal{M} \not\models políčko(x) [e(x/a1)]$

I will choose a case that may not hold.

[Continue](#)

Let's assume that $\mathcal{M} \not\models políčko(x) [e(x/a1)]$

You lose, because $\mathcal{M} \models políčko(x) [e']$, since $x^{\mathcal{M}}[e'] = a1 \in i(políčko)$ where $e' = e(x/a1)$

You could have won, though. Your initial assumption that $\mathcal{M} \models \forall x((figúrka(x) \vee políčko(x)) \rightarrow (farba(x) = biela \vee farba(x) = čierna)) [e]$ was correct. Find incorrect intermediate answers and correct them! You can use the **Change** link next to your answers for that.

(b) Nová verzia

Obr. 4.1: Porovnanie pôvodného a nového rozhrania Henkinovej-Hintikkovej hry.

Kapitola 5

Testovanie

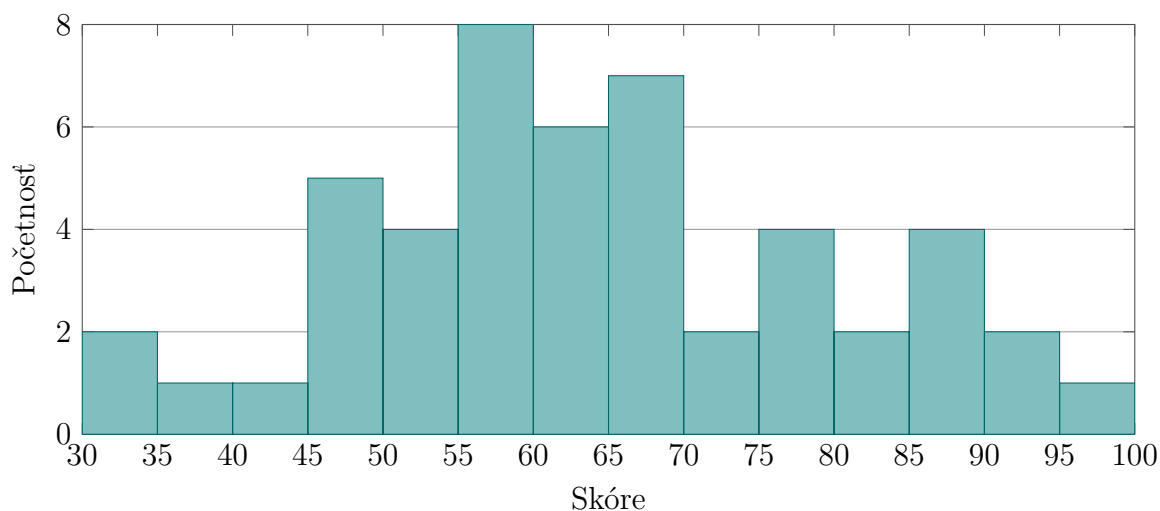
Študenti mali možnosť využívať časť novej funkcionality už od začiatku výučby predmetu Logika pre informatikov v letnom semestri akademického roka 2025/2026. Išlo najmä o grafové editory, maticový editor a databázový editor, ktoré boli v tom čase už z veľkej časti dokončené. Ostatné nástroje sme postupne pridávali v priebehu semestra. Predvoleným editorom bol však väčšinou množinový (textový) pohľad. Na podrobnejšie otestovanie nových editorov sme sa preto rozhodli na jednom z posledných cvičení študentom pri vhodných interpretáciách naopak predvoliť orientovaný graf a editor interpretácií funkcií rozborom prípadov. Tie niekedy slúžili len na vizualizáciu, inokedy bolo pomocu nich potrebné aj upravovať samotnú interpretáciu. Medzi jednotlivými pohľadmi mohli študenti však aj naďalej ľubovoľne prepínať. Príklad jednej štruktúry z cvičenia je na obrázku A.1.¹

Po vypracovaní cvičenia mali študenti možnosť vyplniť krátky dotazník zameraný na získanie spätnej väzby k použiteľnosti pridaných editorov (dotazník možno vidieť na obrázku A.2). Jeho hlavná časť bola zameraná na ohodnotenie použiteľnosti navrhnutých riešení prostredníctvom 10 výrokov prevzatých zo štandardného dotazníka System Usability Scale (SUS) [11]. Pri každom výroku študenti vyjadrovali mieru súhlasu na škále od 1 (rozhodne nesúhlasím) do 5 (rozhodne súhlasím). Na základe odpovedí sa následne vypočítava skóre v rozsahu od 0 do 100, pričom vyššia hodnota znamená lepšiu použiteľnosť.

Dotazník celkovo vyplnilo 49 študentov. Priemerné skóre nám vyšlo 63, čo naznačuje, že študenti pri používaní nových riešení identifikovali určité nedostatky, na ktorých by bolo vhodné ešte zapracovať. Histogram na obrázku 5.1 zobrazuje početnosť jednotlivých SUS skóre.

Najhoršie dopadla otázka zameraná na to, či si študent myslí, že by sa musel naučiť veľa vecí, kým by vedel s novými editormi začať pracovať. S výrokom spolu skôr súhlasilo 11 študentov, 23 skôr nesúhlasilo a 15 odpovedalo neutrálne. Naopak najlepší

¹Pre prehľadnosť obrázka boli odstránené interpretácie niektorých mimologických symbolov.



Obr. 5.1: Histogram jednotlivých SUS skóre z testovania použiteľnosti.

výsledok sme zaznamenali pri otázke, či podľa študenta boli jednotlivé funkcie nových editorov dobre integrované. S výrokom skôr súhlasilo 33 študentov, 4 skôr nesúhlasili a 12 odpovedalo neutrálne. Podobné výsledky sme zaznamenali aj pri ďalších otázkach tohto typu. Rozdiel si vysvetľujeme tak, že študenti nemali často problém s tým, ako jednotlivé editory fungujú, ale skôr s pochopením alebo objavením ich jednotlivých možností. Tento problém sme sa snažili ďalej zmierniť pridaním lepších vysvetliviek k ovládacím prvkom a zrozumiteľnejšou komunikáciou niektorých stavov editorov.

Na konci dotazníka mohli študenti uviesť konkrétne problémy, na ktoré narazili, ako aj prípadné návrhy na ich zlepšenie. Často sa opakovala pripomienka na nadmerné množstvo vnorených rolovateľných prvkov v rámci celej aplikácie Logického pracovného zošita. Je to spôsobené najmä tým, že vertikálne možno rolovať nie len celý hárok, ale aj obsah vložených aplikácií. Grafové editory k tomuto navyše pridali ďalší element reagujúci na rolovanie, keďže pri umiestení kurzora nad zobrazenie grafu je možné jeho obsah zväčšovať alebo zmenšovať. Pri hľadaní vhodného riešenia nás oslovila spätná väzba jedného študenta. V nej navrhol, aby bolo rolovanie v grafoch možné len v prípade, ak je zároveň stlačené pridružené tlačidlo (napr. **Ctrl**), podobne ako je to pri vložených zobrazeniach máp Google. Všetky grafové pohľady sme doplnili o takéto správanie.

Niektorí študenti mali problém aj s tým, že pri pridávaní viacerých nových prvkov domény sa im prislúchajúce vrcholy v zobrazení orientovaného grafu vzájomne prekryvali, čo pôsobilo mätúco. Tento problém sme vyriešili náhodným rozmiestnením nových vrcholov.

Na podnet jedného študenta sme sa tiež rozhodli upraviť editor interpretácií funkcií rozborom prípadov tak, aby pri určovaní podmienky bolo možné zadávať viacero prvkov, nielen jeden konkrétny. Vďaka tomu je tvorba podmienok v určitých situáciách

jednoduchšia, keďže nie je potrebné definovať samostatný prípad pre každú hodnotu podmienky. Novú verziu editora interpretácií funkcií rozborom prípadov možno vidieť na obrázku 5.2. Editor sme navyše vylepšili aj pridaním správy „exhausted on $\langle argument funkcie \rangle$ “, ktorá značí, že pre daný argument už boli špecifikované podmienky na rovnosť s každým prvkom domény. Ak však ešte neboli, používateľ teraz môže umiestnením kurzora nad text „is any other“ vo všeobecnej podmienke pre daný argument zistiť, ktoré prvky sú týmto označením myslené.

$$i(f)(x, y, z) = \left\{ \begin{array}{l} \text{E} \quad \text{if } x \in \{ A, B, C \}, y \in \{ D, E, B \} \quad + \downarrow \quad \text{trash} \\ \text{B} \quad \text{if } x \in \{ A, B, C \}, y \in \{ A, C \} \quad + \downarrow \quad \text{trash} \\ \text{exhausted on } y \\ \text{C} \quad \text{if } x \in \{ E, D \}, z \in \{ A, D \} \quad + \downarrow \quad \text{trash} \\ \quad \text{if } x \in \{ E, D \}, z \in \{ B, C, E \} \\ \text{D} \quad \text{if } x \in \{ E, D \}, z \text{ is any other} \quad + \downarrow \\ \text{exhausted on } x \end{array} \right.$$

Obr. 5.2: Nová verzia editora interpretácií funkcií rozborom prípadov.

Veríme, že spomínané zmeny pomohli odstrániť problémy, ktoré študenti odhalili pri používaní nových editorov. Cieľom ďalšieho testovania aplikácie by určite bolo dosiahnuť lepšie SUS skóre, no v rámci predmetu už naň nie je priestor. Nové vylepšenia a ich prípadné testovanie preto ponechávame na budúci vývoj aplikácie.

Záver

Podarilo sa nám vytvoriť grafové editory obohacujúce aplikáciu Prieskumník štruktúr o nové možnosti tvorby interpretácií predikátov a funkcií. Snažili sme sa pri ich návrhu dosiahnuť v prvom rade prehľadnú a zároveň prispôsobiteľnú vizualizáciu. Zaoberali sme sa nápadmi, ako premeniť rôzne grafové reprezentácie do interaktívnej podoby a spríjemniť ich používanie rôznymi vylepšeniami. Postupne sme tiež pridávali ďalšie užitočné nástroje. Reimplementovali sme maticový a databázový editor z predchádzajúcej verzie aplikácie a prispôbili ich našim špecifickým požiadavkám. Editorom interpretácií funkcií rozborom prípadov sme uľahčili tvorbu veľkých definícií funkcií a pridali sme aj novú sekciu aplikácie pre definovanie dopytov na splniteľnosť formúl.

Ďalej sme popísali integráciu jednotlivých riešení do existujúcej aplikácie, zabezpečili jej kompatibilitu s Logickým pracovným zošitom a nakoniec sa zamerali na vylepšenia súvisiace s Henkinovou-Hintikkovou hrou.

Aplikáciu sa podarilo dostať pomerne skoro do stavu, v ktorom mohla byť nasadená do výučby. Študenti mali preto počas celého semestra príležitosť využívať postupne pribúdajúcu funkcionálnu pri riešení cvičení a domácich úloh. V závere výučby sme jedno z cvičení využili na podrobnejšie testovanie nových nástrojov a následne sme prostredníctvom dotazníka zbierali spätnú väzbu od študentov. Na jej základe sme do aplikácie zapracovali viacero zmien.

Pri jednotlivých nástrojoch je však určite stále priestor na zlepšenie. Jedným z potenciálnych nápadov je pri zobrazení bipartitného grafu umožniť separátne filtrovanie zobrazenej domény oboch skupín. V súčasnosti je totiž množina vrcholov v oboch skupinách identická, čo často neposkytuje dostatočnú flexibilitu na dosiahnutie uspokojivého zobrazenia. Navrhnuté riešenie by sa malo dať využiť aj v maticovom editore na separátne filtrovanie stĺpcov a riadkov.

Rozhranie editora interpretácií funkcií môže zas novým používateľom prísť na prvý pohľad neintuitívne, čo by sa dalo vylepšiť zmenou jeho rozhrania, najmä v oblasti pridávania novej podmienky. Ďalším potenciálnym vylepšením je umožniť konfiguráciu farebnej palety používanej pri vizualizácii unárnych predikátov alebo optimalizovať implementáciu Henkinovej-Hintikkovej hry zmenou spôsobu vyhodnocovania formúl.

Vývojom tejto aplikácie som sa naučil používať nové technológie a prehĺbil znalosť tých, s ktorými som už mal predošlé skúsenosti. Asi najväčšou výzvou z tohto hľadiska

bola pre mňa práca s knižnicou Redux, keďže som s ňou nikdy predtým nepracoval a pri implementovaní riešení bolo jej hlbšie porozumenie kľúčové. Cenná bola aj skúsenosť pocitu zodpovednosti za vykonávanými zmenami počas toho, ako bola aplikácia už nasadená do výučby, keďže som nechcel študentom ani učiteľom spôsobiť zbytočné komplikácie prípadnými nedostatkami.

Literatúra

- [1] *React Diagrams: A flow & process orientated diagramming library*, 2026. Dostupné z <https://projectstorm.cloud/react-diagrams>.
- [2] *React Flow: A library for building node-based UIs*, 2026. Dostupné z <https://reactflow.dev>.
- [3] *Reagraph: WebGL Graph Visualizations for React*, 2026. Dostupné z <https://reagraph.dev>.
- [4] *Redux: A JS library for predictable and maintainable global state management*, 2026. Dostupné z <https://redux.js.org>.
- [5] *XFlow: React libraries for node-based UIs*, 2026. Dostupné z <https://xyflow.dev>.
- [6] *Zod: TypeScript-first schema validation with static type inference*, 2026. Dostupné z <https://zod.dev>.
- [7] Miroslav Baluch. Prieskumník grafových štruktúr pre logiku prvého rádu. Bakalárska práca, Univerzita Komenského v Bratislave, Fakulta matematiky, fyzika a informatiky, 2020.
- [8] David Barker-Plummer, Jon Barwise, and John Etchemendy. Tarski's world: Revised and expanded. 2007.
- [9] David Barker-Plummer, Jon Barwise, and John Etchemendy. *Language, proof, and logic*. Center for the Study of Language and Information/SRI, 2011.
- [10] Jon Barwise. An introduction to first order logic. In Jon Barwise, editor, *Handbook of Mathematical Logic*, volume 90. Elsevier, 1977.
- [11] John Brooke et al. SUS-a quick and dirty usability scale. *Usability evaluation in industry*, 189(194), 1996.
- [12] Milan Cifra. Prieskumník sémantiky logiky prvého rádu. Bakalárska práca, Univerzita Komenského v Bratislave, Fakulta matematiky, fyzika a informatiky, 2018.

- [13] Jozef Filip. Reimplementácia prieskumníka štruktúr pre logiku prvého rádu. Bakalárska práca, Univerzita Komenského v Bratislave, Fakulta matematiky, fyzika a informatiky, 2025.
- [14] R.P. Grimaldi. *Discrete and combinatorial mathematics: an applied introduction*. Addison-Wesley Pub. Co., 1985.
- [15] Ján Kľuka, Ján Mazák, and Jozef Šiška. Logika pre informatikov a Úvod do matematickej logiky: Poznámky z prednášok. 2025.
- [16] Meta Platforms, Inc. *React: A JavaScript library for building user interfaces*, 2026. Dostupné z <https://react.dev>.
- [17] Microsoft. *TypeScript: Typed JavaScript at Any Scale*, 2026. Dostupné z <https://www.typescriptlang.org>.
- [18] Matej Mok. Interaktívny pracovný zošit pre výučbu logiky pre informatikov. Bakalárska práca, Univerzita Komenského v Bratislave, Fakulta matematiky, fyzika a informatiky, 2022.
- [19] Vítězslav Švejdar. *Logika: neúplnosť, složitost a nutnost*. Academia-nakladatelství Akademie věd ČR, 2002.
- [20] Paul Tol. Qualitative color schemes. *Paul Tol's notes. Colour schemes and templates*, 2021.
- [21] Richard Tóth. Henkinova-hintikkova hra v prieskumníku štruktúr. Bakalárska práca, Univerzita Komenského v Bratislave, Fakulta matematiky, fyzika a informatiky, 2021.

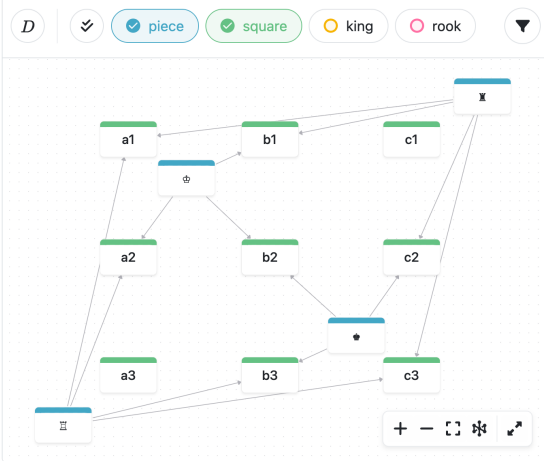
Príloha A: Materiály k testovaniu

Language $\mathcal{L}$?
 $\mathcal{C}_{\mathcal{L}} = \{ \text{white, black} \}$
 $\mathcal{P}_{\mathcal{L}} = \{ \text{piece/1, square/1, king/1, rook/1, may_enter/2, may_take/2} \}$
 $\mathcal{F}_{\mathcal{L}} = \{ \text{on/1, clr/1} \}$
Structure $\mathcal{M} = (D, i)$?

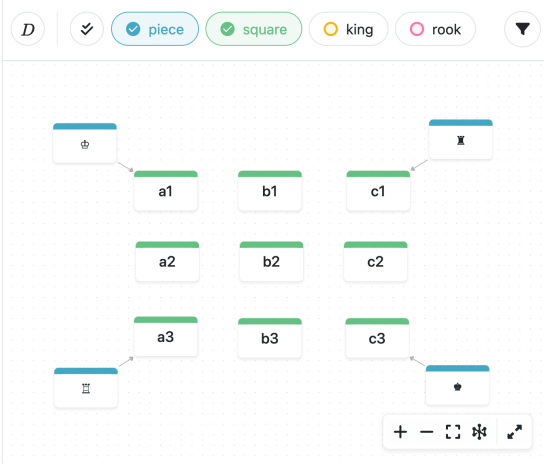
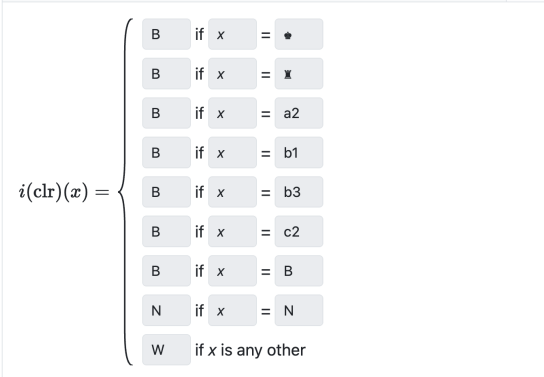
Domain

 $D = \{ \clubsuit, \spadesuit, \heartsuit, \blacksquare, a1, a2, a3, b1, b2, b3, c1, c2, c3, W, B, N \}$

Predicates interpretation

 $i(\text{may_enter})$ / Oriented Graph


Functions interpretation

 $i(\text{on})$ / Oriented Graph

 $i(\text{clr})$ / Case Tree Editor
Truth of formulas in $\mathcal{M}$?

Prettify formulas

 $\varphi_1 \equiv (\forall x (\text{piece}(x) \rightarrow \text{square}(\text{on}(x))) \wedge \forall x ((\text{king}(x) \vee \text{rook}(x)) \rightarrow \text{piece}(x)))$
 $\mathcal{M} \models/\neq? \quad \varphi_1[e]$

Verify

 $\varphi_2 \equiv \forall x ((\text{piece}(x) \vee \text{square}(x)) \rightarrow (\text{clr}(x) \neq \text{white} \vee \text{clr}(x) \neq \text{black}))$
 $\mathcal{M} \models/\neq? \quad \varphi_2[e]$

Verify

 $\varphi_3 \equiv \forall x \forall y (((\text{square}(x) \wedge \text{piece}(y)) \wedge \text{on}(y) \neq x) \rightarrow \forall z (((\text{piece}(z) \wedge \text{on}(z) \neq x$
 $\mathcal{M} \models/\neq? \quad \varphi_3[e]$

Verify

 $\varphi_4 \equiv (\forall x (\text{piece}(x) \rightarrow (\neg \text{square}(x) \wedge \neg \exists y (\text{clr}(y) \neq x))) \wedge \forall x (\text{square}(x) \rightarrow \neg \exists y c$
 $\mathcal{M} \models/\neq? \quad \varphi_4[e]$

Verify

 $\varphi_5 \equiv \forall x \forall y ((\text{piece}(x) \wedge \text{piece}(y)) \rightarrow ((\neg \text{clr}(x) = \text{clr}(y) \wedge \text{may_enter}(x, \text{on}(y))$
 $\mathcal{M} \models/\neq? \quad \varphi_5[e]$

Verify

+ Add

Queries in $\mathcal{M}$?

+ Add

Obr. A.1: Štruktúra z cvičenia zameraného na testovanie alternatívnych editorov.

Nasledujúce výroky slúžia na vyjadrenie Vašich skúseností s používaním alternatívnych editorov. Pri každom prosím označte mieru svojho súhlasu na škále od 1 do 5, kde:

- 1 = rozhodne nesúhlasím
- 5 = rozhodne súhlasím

1.	Myslím si, že by som alternatívne editory chcel/a používať často.	
2.	Alternatívne editory sa mi zdali zbytočne zložité.	
3.	Myslím si, že alternatívne editory boli jednoduché na použitie.	
4.	Myslím si, že by som potreboval/a pomoc technicky zdatnej osoby, aby som vedel/a tieto alternatívne editory používať.	
5.	Jednotlivé funkcie alternatívnych editorov boli podľa mňa dobre integrované.	
6.	Myslím si, že v alternatívnych editoroch bolo príliš veľa nekonzistentností.	
7.	Predpokladám, že väčšina ľudí by sa naučila používať tieto alternatívne editory veľmi rýchlo.	
8.	Tieto alternatívne editory sa mi zdali veľmi ťažkopádne na používanie.	
9.	Pri používaní alternatívnych editorov som sa cítil/a veľmi sebaisto.	
10.	Musel/a som sa naučiť veľa vecí, kým som vedel/a s týmito alternatívnymi editormi začať pracovať.	

Sem môžete uviesť konkrétne pripomienky k alternatívnym editorom a prípadne aj návrhy na ich zlepšenie.

.....

Obr. A.2: Dotazník SUS pre alternatívne editory.